

QKF-Certifier: observation kernels and width-independent word proofs

Leonid Shcherbakov

22 September 2026

Preprint, version 3.0

Abstract

QKF studies how incomplete observations determine computations through forced rows, their kernels, and compatible quotients. This paper develops that viewpoint into a certificate architecture for word programs. A consumer specifies the distinctions its proof needs; checked native facts label those distinctions; finite algebraic proofs establish the required action; and independently checked certificates connect that action to the source and the consumer.

We retain the exact finite relation-matrix kernel, the complete nine-row characterization of coordinatewise KnownBits, and the signed mask-interval case studies. A descending pass has a 49-state certificate for an exact masked maximum; an ascending pass factors three physical phases into two observations and supports a conditional lower-preservation contract. The revised architecture adds automatic observation selection within a finite source-derived language, a source-bound signed runtime, conditional residual coverage, and a separate source-to-consumer route that can prove a requested equality without first constructing the whole residual carrier. A pure-row checker computes initial forced domains, while a typed ground-DAG checker supplies chronological equality proofs and finite separating models. Checked closed lemmas, prepared contexts, and direct proof emission support reusable, executable summaries.

Lean 4 checks the earlier word fragments and, in two new packages, soundness of the decoded typed positive ground calculus and chronological composition of checked lemmas under explicit native premises. JSON decoding, source frontends, native source rules, and runtime correspondence remain outside those general formal results. Retained external results include 11 of 39 complete Nice to Meet You programs and 104 of 411 components; later development experiments do not enlarge those denominators. Controlled measurements show substantial reductions in preparation and proof-emission overhead, while QKF’s own simpler ordinary-SDK route remains the faster registered baseline. These results separate the value of the certificate architecture from an unestablished performance advantage of its query backend.

1 Introduction

A word program may inspect a count, use a shift, propagate a carry, and finally return only a few facts about its result. A proof that reconstructs every intermediate value can retain substantially more information than the consumer needs. QKF asks which observations determine this particular action, and which of their values are forced by the supplied information and the operations already justified on the carrier.

This is Paper III of a series by Leonid Shcherbakov. Paper I develops initial semantics for incomplete actions, forced domains, kernel saturation, and completion [1]. Paper II studies equality visibility through finite ground interfaces and distinguishes congruence-proof depth from the size of sequential rewriting contexts [2]. The present paper connects these distinctions

to executable proof objects. Native semantics supplies the facts to which the algebra applies; a quotient is useful only when its labels and its interaction with the consumer have been justified.

The first version described an exact finite algebraic kernel and a KnownBits certifier whose final language was coordinatewise: nine one-bit rows suffice for a theorem at every positive width. Version 2 added contextual observations, labeled whole-word reconstruction, signed mask–interval carriers, and a Lean pilot. Version 3 develops two complementary paths. One derives a sufficient finite source interface and exposes its checked action. The other obtains native facts for an independently specified consumer and proves the resulting closed equality directly. Their common output is a scoped, replayable guarantee; their discovery and execution costs need not be the same.

Contributions and stable claims. Claims K1–K10 retain their meanings: finite ground closure and its sparse contract (K1–K3), coordinatewise source replay and semantic classification (K4–K5), sufficient labeled observations and finite gluing (K6–K7), the upper and lower masked-bound contracts (K8–K9), and the original Lean pilot (K10). This revision adds executable initial pure-row semantics, restricted automatic source-interface inference, chronological typed ground proofs, checked-lemma composition, and a summary SDK. The new formal results concern the decoded positive proof calculus and its composition; they do not enlarge K10 into a proof of the entire software stack.

Layer	Computation	Evidence in this revision
Finite algebra	Forced relations, initial rows, equality visibility	Mathematical arguments, proof/model replay, independent finite checks.
Coordinatewise CLI	Source normalization and nine-row targets	Stable package contract and retained regressions.
Source interfaces	Observation inference, coverage, row execution	Source-bound packages and independent target products.
Consumer summaries	Native facts, ground DAGs, checked lemmas	Scoped SDK objects, portable proofs, controlled route comparisons.
Lean projects	Word fragments, typed ground soundness, lemma composition	Kernel-checked proofs with explicit assumptions.

The installed command-line package does not automatically acquire every research frontend. Paper II does not confer a minimum proof-depth guarantee on an ordinary normalization trace. Likewise, a successful factorization is not a consumer proof, and a finite separating model of insufficient premises is not necessarily a counterexample to the source program. The explicit maps between these layers are part of the specification.

Version and source identity. This is manuscript version 3.0, dated 22 September 2026, accompanying the 0.3.0a1 research-alpha release preparation. The technical and experimental snapshot is the merged main revision through PR51:

eca90f9252da4480d77c1f8cb4b7062470c57b89.

The repository is <https://github.com/Len989/QKF-Certifier>. Artifact links in the new sections resolve to this immutable revision; individual reports additionally identify the engine that was measured. Earlier coordinatewise results retain their original baseline `f93561682319e8b911ed32c01583f2a496dc59fc`. The release manifest identifies the packaged revision and files. Manuscript version, software version, certificate schema, and semantic

contract are separate identifiers. The prior Paper III deposit is version 2.0 [4]; no new version DOI is presumed here.

The immediate research objective is testable: infer adequate interfaces within explicit source contracts, obtain only the facts a consumer needs, preserve their dependencies across reuse, and reduce total delivery and checking cost. The broader objective of handling substantially wider verification tasks remains open. The internal ordinary-SDK route is part of QKF: its favorable results inform implementation choices within this architecture.

2 The finite partial core inherited from I and II

Let Σ be a finite finitary signature, E a finite set of ground equations, and $\mathcal{Q}$ a finite set of query terms. Constants have depth zero. Put

$$\begin{aligned} S &= \text{Sub}(E) \cup \text{Sub}(\mathcal{Q}), & S_D &= \{u \in S : \text{dep } u \leq D\}, \\ E_D &= \{s = t \in E : \max(\text{dep } s, \text{dep } t) \leq D\}. \end{aligned}$$

Let $\approx_D^S$ be congruence closure on the represented applications in S_D . Write $\equiv_D$ for the bounded *congruence-proof* equality of Paper II and $\equiv_E$ for full ground equality.

Proposition 2.1 (Partial core and evaluation, K1). *The quotient $\mathbf{P}_D = S_D / \approx_D^S$ is a well-defined partial algebra: a tuple has an operation value when it is represented by an application node, and its value is the parent class. Every total $M \models E_D$ induces an operation-preserving map*

$$\eta_M : \mathbf{P}_D \rightarrow M, \quad [u] \mapsto \llbracket u \rrbracket_M.$$

For active query terms s, t , their classes coincide exactly when $s \equiv_D t$. At $H = \max\{h(E), d_{\mathcal{Q}}\}$ this is equivalent to $E \vdash s = t$.

Proof. Congruence closure identifies the parents of any two represented tuples with componentwise equal arguments, so the partial operations are well defined. Every generating equation holds in M , and equality is preserved by its operations; hence evaluation factors through the quotient and preserves represented operations. The equality assertions are Paper II, Theorems 4.1 and 3.1, respectively. Empty query depth and empty axiom depth are taken to be zero. $\square$

This core precedes any choice of values for unrepresented tuples. A finite separating total model is obtained by assigning a default value to each unrepresented operation tuple and adding a default element to every empty active sort. That totalization is a certificate-producing construction, not additional canonical information in $\mathbf{P}_D$.

A positive derivation shows that two classes must merge. Conversely, a total model of E_D separating their labels shows that they cannot merge at that horizon. Paper II, Proposition 4.3 and Corollary 4.5, combine a chronological proof DAG at the first complete threshold with such a lower model. Their resource λ_E bounds all terms in a congruence proof. The different sequential resource λ_E^{rw} bounds whole intermediate rewrite terms. Neither is asserted to equal the number of normalization steps in the public certifier.

2.1 The same action example in all three papers

For the carrier $A = \{0, 1, 2, 3\}$, use the operation table

f	0	1	2	3
0	0	2	0	0
1	1	2	1	3
2	0	2	2	0
3	3	0	3	0

and one operator b , with $r_i = \alpha(b, i)$ and observations $r_0 = 0, r_2 = 2, r_3 = 0$. Include the native diagram and named-input compatibility. Paper I, Example 6.2, and Paper II, Section 5.1, give

$$\begin{aligned}\mathcal{I}/\equiv_1 &= \{\{0, r_0, r_3\}, \{1\}, \{2, r_2\}, \{3\}, \{r_1\}\}, \\ \mathcal{I}/\equiv_2 &= \{\{0, 2, r_0, r_1, r_2, r_3\}, \{1\}, \{3\}\}.\end{aligned}$$

For example, $f(0, 1) = 2$ and $f(3, 1) = 0$ yield $r_2 = f(r_0, r_1) = f(r_3, r_1) = r_0$. Likewise $f(1, 0) = 1$ and $f(1, 3) = 3$ yield $r_1 = r_3$. The canonical maps in Proposition 2.1 thus recover the five-to-three class change on exactly the named interface described in Paper I, Theorem 6.6.

This is an algebraic example, not a KnownBits program. Its role in III is to fix the meaning of a forced quotient and a finite interface before introducing specialized representations. Quotient-feedback rounds, syntactic depth, proof length, and semantic table size remain different quantities throughout.

3 Exact relation-matrix closure

3.1 Ground specification and labelled copies

Let A be a nonempty finite algebra, $|A| = n$, whose native operations have positive arity. Add a fresh binary symbol τ and observations $\Gamma \subseteq A^2 \times A$. The ground theory E_τ contains the complete named native diagram, the observed equations $\tau(a, b) = y$, and separate named-input compatibility. For each native k -ary g , include

$$\begin{aligned}\tau(g(a_1, \dots, a_k), b) &= g(\tau(a_1, b), \dots, \tau(a_k, b)), \\ \tau(b, g(a_1, \dots, a_k)) &= g(\tau(b, a_1), \dots, \tau(b, a_k)).\end{aligned}$$

All displayed arguments are carrier names. No universal identity for g on unnamed elements is imposed. The constants naming A are not nullary native operations required to be preserved by every slice.

Use copies $\Lambda = \{*\} \cup \{R_b : b \in A\} \cup \{C_a : a \in A\}$. A label (i, x) denotes

$$\ell_*(x) = x, \quad \ell_{R_b}(x) = \tau(x, b), \quad \ell_{C_a}(x) = \tau(a, x).$$

Two slice labels can represent one physical cell. Each relation $R_{ij} \subseteq A^2$ records proposed equalities between the labels in copies i and j .

3.2 The closure rules

Initialize diagonal reflexivity, physical-cell facts $(a, b) \in R_{R_b, C_a}$, and the facts $(a, y) \in R_{R_b, *}$ and $(b, y) \in R_{C_a, *}$ for every observation. Saturate the following rules:

- (R1) Converse: $(a, b) \in R_{ij}$ implies $(b, a) \in R_{ji}$.
- (R2) Composition: $(a, b) \in R_{ij}$ and $(b, c) \in R_{jk}$ imply $(a, c) \in R_{ik}$.

(R3) Native compatibility: from $(a_l, b_l) \in R_{ij}$ for all $1 \leq l \leq k$, infer

$$(g^A(a_1, \dots, a_k), g^A(b_1, \dots, b_k)) \in R_{ij}.$$

(R4) Local feedback: $R_{**} \subseteq R_{ii}$ for every copy i .

(R5) Context feedback: $b R_{**} d$ implies $(x, x) \in R_{R_b, R_d}$ and $(x, x) \in R_{C_b, C_d}$ for every x .

Let R^* be the least closed matrix containing the initial facts.

Theorem 3.1 (Exact relation kernel, K2). *A fair saturation that returns only at a common fixed point terminates and computes R^* . For all interface labels,*

$$(x, y) \in R_{ij}^* \iff E_\tau \vdash \ell_i(x) = \ell_j(y).$$

*In particular $R_{**}^* = \theta_{E_\tau}^A$. The carrier values forced for $\tau(a, b)$ are exactly $\{y : (a, y) \in R_{R_b, *}^*\}$; a nonempty such set is one central-congruence class.*

Proof. There are $m = 1 + 2n$ copies and at most $m^2 n^2$ relation facts. Every productive rule application adds a fact. Finite monotone saturation therefore reaches the least fixed point; a worklist or complete-pass implementation must stop when no new fact is available.

For soundness, converse and composition express equality, and physical-cell facts identify the two names of the same term. Each copy preserves every named native operation by the diagram or compatibility equations. Thus native compatibility is sound between any pair of copies. Local and context feedback are congruence of τ in its varying and fixed arguments. All added facts are consequently ground consequences of E_τ .

For completeness, take any closed matrix R . On $V = \Lambda \times A$ put $(i, a) \sim (j, b)$ precisely when $(a, b) \in R_{ij}$. Reflexivity, converse, and composition make this an equivalence. Set $M = V/\sim$. On tuples represented in one copy, define

$$g^M([(i, a_1)], \dots, [(i, a_k)]) = [(i, g^A(a_1, \dots, a_k))].$$

If another representation of the same tuple uses copy j , native compatibility identifies the two proposed results. This defines a partial operation on classes; assign a fixed default class to all other tuples. Interpret carrier names by their central classes and put

$$\tau^M([(*, a)], [(*, b)]) = [(R_b, a)].$$

Local feedback makes the value independent of the representative of a ; context feedback does the same for b . Physical-cell facts give the equivalent column interpretation. Complete all remaining τ -tuples by a default class.

The named diagram, observations, and both compatibility families now hold by construction. Each interface label evaluates to its own class, so distinct R -classes are separated by a total model of E_τ . Applying this construction to R^* proves completeness. The assertion about forced values follows by transitivity with central equality. $\square$

The bound counts possible facts, not the running time of an inefficient repeated composition scan. For fixed native arities an explicit implementation is finite and polynomial in the explicitly represented carrier and rule-instance space; no polynomial bound in a succinct bit-width parameter follows. A native nullary operation would require additional zero-premise cross-copy facts and is excluded from this specification.

Reference algorithm. Initialize the labelled relations; repeatedly apply feedback, native compatibility, and full relational composition, inserting converse facts with each insertion. Return only when a complete pass makes no change. The supplement implements this deliberately transparent oracle and compares its entire carrier/cell partition with independent ground subterm closure. A finite resource limit must return unresolved, never an unfinished matrix presented as exact.

3.3 Certificates and two necessary closure mechanisms

Corollary 3.2 (Checking a proposed exact partition). *A proposed interface partition is exact if (i) generators of each of its blocks have checked derivations from the initial facts and rules, and (ii) the partition contains every initial fact and is closed under every rule.*

Proof. The derivations give $R \subseteq R^*$. Closure and the initial facts give $R^* \subseteq R$, by leastness. Equivalently, the first check excludes unjustified equalities and the second supplies the separating model from Theorem 3.1. $\square$

This algebraic certificate contract is separate from `qkf-rewrite-v3`, which certifies word-expression rewrites.

Example 3.3 (Composing generators is insufficient). In the one-bit KnownBits join-semilattice, write U for unknown and Z for known zero, so $U \vee_A Z = U$. Let $G = \{(U, U)\}$ and $H = \{(U, Z), (Z, U)\}$. The join-closure of H also contains (U, U) . Hence the composition of the two closed relations contains both (U, Z) and (U, U) . Composing only the displayed generators gives $\{(U, Z)\}$, whose join-closure still omits (U, U) . Rule R2 must quantify over the full represented relations, including derived intermediate witnesses.

Example 3.4 (Feedback determines additional cells). Take the two-element join-semilattice and supply both $\tau(0, 0) = 0$ and $\tau(0, 0) = 1$. The central carrier collapses. With feedback, all four named input cells become equal to that central class. If feedback is omitted while the other rules are saturated, only the observed cell is connected to the center. This finite example is checked in the supplement.

The same model proof extends to a fixed p -ary τ using $1 + pn^{p-1}$ copies, one for each varying coordinate and fixed context. All views of a physical cell must be identified, and feedback must cover every context coordinate. The implementation and differential results reported here concern the binary case.

4 Sparse semilattice representations

Suppose now $A = (A, \vee_A)$ is a finite join-semilattice and τ is binary. The relation matrix can be represented by global equality components of labelled endpoints instead of storing every R_{ij} explicitly. A missing physical cell still denotes a two-endpoint component, one in its row and one in its column. It is implicit, not absent.

For the current central quotient $Q = A/\theta$, materialize all central endpoints. Materializing either view of a cell also materializes its other view and identifies them. For each used copy i , maintain its exact local congruence $\kappa_i \in \text{Con}(Q)$ and materialize every affected nontrivial local class. This guarantees that an unmaterialized cell has not silently acquired further equalities.

Lemma 4.1 (Compatibility coverage). *After exact local closure, all binary native-compatibility obligations are covered by:*

- (i) *each pair of distinct explicit global components, compared in every copy in which both occur;*
- (ii) *each explicit component and implicit physical cell, compared across that cell's row and column when both occur in the explicit component.*

Repeated components and pairs of distinct implicit cells create no further cross-copy obligation.

Proof. Representatives of one component inside one copy are interchangeable modulo κ_i . This reduces two distinct explicit components to one pair of representatives per common copy.

Repeating one component gives its own class by idempotence; local closure handles different representatives of that component. Commutativity allows unordered component pairs.

An implicit component occurs in exactly one row and one column. Thus its only possible nontrivial comparison with an explicit component is (ii). Two distinct implicit cells cannot share both their row and column, so they have at most one common copy and create no cross-copy identification. Repeating one implicit cell is again covered by idempotence. Local compatibility in a single copy is already included in exact local congruence closure. $\square$

Theorem 4.2 (Sparse refinement contract, K3). *An algorithm that preserves the representation above, fairly saturates the two cases of Lemma 4.1, and performs full central quotient feedback computes the same interface partition as Theorem 3.1, provided it reports success only after a common fixed point is reached.*

Proof. Every materialization, local merge, and component-compatibility merge is justified by a relation-matrix rule. When central conflicts occur, close them as a native congruence in A , replace the carrier by the resulting quotient, and restart from the projection of the original observations. These are sound central consequences and implement both local and context feedback.

At a fixed point without central conflicts, expand the implicit cells conceptually. The local congruences, Lemma 4.1, and the quotient representation satisfy every rule of Theorem 3.1. Soundness and leastness therefore identify the result with R^* . At a fixed Q , the number of possible labelled endpoints and equality merges is finite. The central congruence can change strictly at most $n - 1$ times. An uncapped fair monotone implementation consequently terminates; a prematurely exhausted engineering budget must return unresolved. $\square$

Algorithmic order. Start with the diagonal central congruence. Build Q and projected observations; close mirrors and local congruences, then both compatibility cases, repeating until stable or a central conflict is found. If a conflict appears, enlarge θ by its native congruence closure and rebuild over Q . Return only after a stable round with no conflict. This specification exposes the conditions an optimized representation must preserve, rather than assuming that a choice of data structure establishes completeness.

4.1 Principal generators and a width-dependent carrier

For $s \leq u$ in a join-semilattice, the comparably generated principal congruence satisfies

$$\text{Cg}(u, s) = \Delta \cup \{(a, b) : s \leq a, b, u \vee_A a = u \vee_A b\}. \quad (4.1)$$

We use this established characterization [9, Theorem 2.1], together with

$$\text{Cg}(x, y) = \text{Cg}(x \vee_A y, x) \vee \text{Cg}(x \vee_A y, y).$$

Grouping the elements $a \geq s$ by $u \vee_A a$ supplies generators for each class in (4.1). Inserting these pairs directly into a persistent local disjoint-set structure computes its join with the new principal congruence; rebuilding a separate full-carrier structure for each seed is unnecessary.

On nonempty KnownBits values at width w , native join retains the bits known in both arguments:

$$(Z, O) \vee_A (Z', O') = (Z \& Z', O \& O').$$

There are 3^w carrier values. Above a fixed element, the original discrete carrier consists of submasks of its known literals, so at most 2^w values need inspection for a comparable principal generator. After a nontrivial quotient, a general quotient-carrier scan remains a valid option.

These observations concern representation cost in a width-dependent finite algebra; they do not give a polynomial-time algorithm in w .

The historical component-pair C++ backend implements these ideas and has substantial differential evidence. Its source includes a 100-pass loop without a distinct exhaustion result. Therefore Theorem 4.2 is a proved representation contract, not an unconditional correctness theorem for that exact historical file. The public Python certifier has its own explicit resource-limit outcomes and does not call this backend.

4.2 Algebraic compatibility does not establish concrete soundness

A constant τ preserves a positive-arity semilattice operation separately in both arguments. Choosing its constant to be a wrong concrete answer gives an algebraically compatible but unsound transformer. Consequently relation closure cannot turn arbitrary sampled examples into a soundness proof for an unrestricted candidate language.

Nor does every optimal transformer preserve this native join. At width two, let $a = 00$, $a' = 11$, and $b = 01$ denote exact words. Then $a \vee_A a' = ??$, and the best modular-add transformer satisfies

$$\text{Add}^\#(a \vee_A a', b) = ??, \quad \text{Add}^\#(a, b) \vee_A \text{Add}^\#(a', b) = 01 \vee_A 00 = 0?.$$

Thus separate join compatibility is not a general axiom of optimal KnownBits addition. The word-certification route below uses the actual expression semantics instead of importing such an axiom.

5 The complete finite semantics of the accepted word fragment

5.1 KnownBits and the coordinatewise language

For $w \geq 1$, write $B_w = \{0, \dots, 2^w - 1\}$ and let

$$\text{KB}_w = \{(Z, O) \in B_w^2 : Z \& O = 0\}.$$

Its concretization is

$$\gamma_w(Z, O) = \{x \in B_w : x \& Z = 0, x \& O = O\}.$$

Inputs are nonbottom: every bit is known zero, known one, or unknown. We use $\mathbb{K} = \{Z, O, U\}$ for these three one-bit states, encoded respectively by $(1, 0)$, $(0, 1)$, and $(0, 0)$. The context distinguishes the state symbols from word masks. An output with conflicting masks has empty concretization and cannot be sound for the nonempty target images considered here.

The accepted mask language has four variables z_a, o_a, z_b, o_b , the constants zero and all-ones, and bitwise AND, OR, XOR, and NOT. A pair $e = (e_Z, e_O)$ consists of two such expressions. The same syntax is interpreted at every width. The ordinary integer constant 1 is not an all-ones constant at widths above one and is therefore outside this final language, as are counts, shifts, comparisons, and arithmetic residuals.

KnownBits and the closely related tnum domain are established abstract domains [7, 8]. We use raw zero/one masks so that the exact interface matches the program's return value. Our class counts below do not identify distinct invalid masks merely because they denote bottom.

Lemma 5.1 (Coordinate evaluation). *For every accepted mask expression e , positive width w , and bit position $j < w$, its output bit depends only on the four input-mask bits at j and equals the one-bit evaluation of e on those bits.*

Proof. Variables and the two constants have the stated property. Each permitted operation applies its Boolean operation independently at each bit. Structural induction gives the result. $\square$

Theorem 5.2 (Nine-row soundness and optimality, K5.1). *Let f_w be the coordinatewise lift of a Boolean binary target f_1 . An accepted pair expression e is sound at every positive width if and only if, for every $(a, b) \in \mathbb{K}^2$,*

$$\{f_1(x, y) : x \in \gamma_1(a), y \in \gamma_1(b)\} \subseteq \gamma_1(e_1(a, b)). \quad (5.1)$$

Replacing inclusion by equality gives a necessary and sufficient criterion for the best KnownBits result at all positive widths. A failed soundness row yields a concrete width-one counterexample.

Proof. Necessity follows by choosing width one. For sufficiency, every valid word input is a product of its one-bit possibilities. By Lemma 5.1, (5.1) at each coordinate includes the corresponding concrete target bit in the output. This proves soundness of the whole word.

For optimality, coordinatewise target evaluation on product inputs has a product image: choices of the input bits at different positions are independent. Each coordinate image is a nonempty subset of $\{0, 1\}$ and is represented exactly by one of Z, O, U . Equality in all rows therefore gives exactly the product image, which is the best domain value. If inclusion fails, the offending row contains concrete bits whose target result violates an output mask, already at width one. $\square$

For the three supported targets the exact table is:

a	b	AND	OR	XOR
Z	Z	Z	Z	Z
Z	O	Z	O	O
Z	U	Z	U	U
O	Z	Z	O	O
O	O	O	O	Z
O	U	U	O	U
U	Z	Z	U	U
U	O	U	O	U
U	U	U	U	U

5.2 An extensional quotient, not a syntactic normal form

Order the input rows lexicographically using (Z, O, U) on each argument, with the second argument varying fastest. Encode the zero and one output bits of row i in positions $2i$ and $2i + 1$:

$$\sigma(e) = \sum_{i=0}^8 (e_Z(i)2^{2i} + e_O(i)2^{2i+1}).$$

This is the exact 18-bit signature reported by `inspect`.

Theorem 5.3 (Complete finite signature, K5.2). *Two accepted pair expressions have equal signatures exactly when they return identical masks at every positive width on every valid input. There are exactly 2^{18} raw-mask semantic classes and 3^9 classes with valid outputs on every valid input. Evaluation of one mask is a Boolean-algebra homomorphism onto the 512-element algebra of Boolean functions on the nine rows.*

Proof. Equality of signatures is equality on the nine rows and lifts coordinatewise by Lemma 5.1; conversely, take width one. To show surjectivity, each valid assignment of the four mask bits has a Boolean minterm using variables and their negations that singles it out among the nine rows. Disjunctions of these minterms define every subset of rows. Thus one mask has 2^9 classes, and two independently chosen masks have 2^{18} . Restricting each row to the three nonconflicting outputs gives 3^9 . Boolean operations are evaluated pointwise, proving the homomorphism assertion. $\square$

The kernel of this evaluation is an exact semantic congruence. It is a second concrete instance of the evaluation-kernel viewpoint from Proposition 2.1, but its semantic models and permitted expressions are specified independently of E_{τ} . The implemented normalizer is not claimed to choose one unique syntax for each signature.

For componentwise mask meet, $(Z, O) \sqcap (Z', O') = (Z \mid Z', O \mid O')$, the signature obeys

$$\sigma(e \sqcap e') = \sigma(e) \text{ OR } \sigma(e'). \quad (5.2)$$

On arbitrary raw outputs this operation can produce conflicting masks. On sound transformers of the same target, the common nonempty concrete image prevents that conflict.

5.3 The exact precision classes

Let $\mathfrak{F}_f$ be the input rows on which the target image is a singleton, annotated by that forced bit. These are the possible correct precision facts. No monotonicity assumption is imposed in the following classification.

Theorem 5.4 (Sound precision lattice, K5.3). *Sound coordinatewise transformer classes for f are in bijection with subsets of $\mathfrak{F}_f$. Every subset is expressible in the accepted language. Meet corresponds to union of precision facts. For AND, OR, and XOR there are respectively 64, 64, and 16 sound classes, and a chain of strict meet refinements starting at all-unknown has at most 6, 6, and 4 strict steps.*

Proof. When the target row image is $\{0, 1\}$, soundness forces output U . When the image is a singleton, the only sound outputs are its correct singleton and U . Thus a sound table chooses exactly which forced facts to retain. Theorem 5.3 gives definability of every such table, and (5.2) gives union. The displayed target table has six, six, and four forced rows. Each strict refinement adds at least one previously absent fact. $\square$

These heights bound semantic precision within the accepted fragment. They do not bound candidate-generation iterations, proof-search cost, or the cost of the best implementation of a class.

Proposition 5.5 (Additional monotonicity). *If transformers must also be monotone for the precision order induced by concretization inclusion, the numbers of sound classes are 26, 26, and 16 for AND, OR, and XOR.*

Proof. A retained forced fact at an input row requires the same fact at every more precise row. For AND, the five zero rows have three minimal rows (Z, Z) , (Z, O) , (O, Z) and two further rows (Z, U) , (U, Z) . Selecting (Z, U) requires the first two minimal rows; selecting (U, Z) requires the first and third. The number of downsets is $8 + 2 + 2 + 1 = 13$, according to whether neither, only the first, only the second, or both further rows are selected. The one row (O, O) forcing one is independently optional, giving 26. OR is dual. All four forced XOR rows are minimal, so every subset is monotone. $\square$

The release computes the 18-bit signature and target soundness/optimality; it does not expose a separate monotonicity flag. The additional classification is a mathematical result checked in the supplement and a possible future interface extension.

6 From source text to a checked all-width verdict

6.1 The source-semantics contract

The source bundle maps caller-chosen labels to UTF-8 program texts. The entry function takes two KnownBits pairs. The supported grammar is a restricted, line-oriented subset of transfer MLIR: single-block typed SSA functions, pure operations, acyclic calls with explicitly supplied definitions, and an actual return instruction. It is not the full MLIR language. Unknown syntax, unresolved calls, recursion, type inconsistencies, and exhausted resource budgets do not produce a successful target verdict.

At width $w \geq 1$, word arithmetic is modulo 2^w . Boolean expressions have separate two-valued semantics. Counts return integers in $[0, w]$, encoded as words; $w < 2^w$ ensures this is possible. Shifts and division use the total semantics specified in Appendix A. In particular, the source operation `transfer.neg` denotes bitwise NOT in this contract. No poison or undefined behavior is introduced. Since every operation is pure and total, discarding an unused computation preserves the returned value.

Lemma 6.1 (Expression construction). *For this grammar and semantics, symbolic evaluation of SSA instructions, expansion of acyclic calls, and projection of the actual return produce an expression e_0 with the same value as the source entry at every positive width on every valid input.*

Proof. Maintain an environment mapping each defined SSA name to an expression with its current semantic value. Input projections initialize the four mask variables. Pair construction and projection preserve the pair semantics. Each primitive instruction adds its specified operation to already-correct operand expressions. A call substitutes the actual argument expressions for the callee’s formal arguments; induction on the acyclic call graph justifies its expansion. Induction on instruction order maintains the environment invariant. Selecting the expression for the actual return gives the result. Totality justifies ignoring expressions that do not contribute to that return. $\square$

The frontend module performs this translation through its `parse_bundle` and `expression` functions. The lemma describes the translation algorithm; correspondence of the Python parser with the documented text grammar is part of the implementation trust described below.

6.2 Local rules and semantic guards

The release has 29 rule identifiers. Most implement direct constant, idempotence, cancellation, count, comparison, and selection laws. Two guards reason about support: `disjoint` establishes that two operands have no common one bits, and `subset` establishes inclusion of one operand’s support in another’s. On coordinatewise expressions they exhaust the nine valid input rows; structural cases use AND restriction and OR inclusion. The `lowbit` guard proves a constant low bit using bitwise laws and parity of modular addition/subtraction.

Lemma 6.2 (Sound guards). *If the release’s support guards return true, their respective disjointness or inclusion assertions hold at every positive width on every valid input. If `lowbit` returns $c \in \{0, 1\}$, the operand has low bit c at every positive width.*

Proof. For coordinatewise guards, Lemma 5.1 lifts the exhaustive one-bit checks. The structural disjointness cases replace an AND-expression by an operand with a superset of its support, so established disjointness is preserved. The subset cases use reflexivity, empty support, full support, inclusion into either branch of OR, and inclusion from either operand of AND.

For the low bit, zero, all-ones, and integer literals have their stated parity. A shift left by the word constant one clears that bit, including at $w = 1$. NOT flips it, AND/OR use their Boolean laws, and XOR, modular addition, and modular subtraction have low bit equal to parity XOR. Each recursive case is therefore sound. $\square$

For example, if $x \& y = 0$, addition has no carries and $x + y = x \mid y$ at every width. If $\text{supp}(x) \subseteq \text{supp}(y)$, setting the leading $\text{countl_one}(x)$ bits of y changes nothing. These are precisely the guarded rules **disjoint-add** and **set-leading-included**. Appendix A gives the complete rule catalogue and proof arguments for the remaining families.

Proposition 6.3 (Local rewrite preservation). *Every applicable rewrite in the mathematical rule system listed in Appendix A preserves value at every positive width on valid KnownBits inputs.*

Proof. The guarded rules use Lemma 6.2. The remaining rules follow from the stated modular, Boolean, order, count, and total division/shift semantics, as checked by the cases in the appendix. In particular, the rules involving zero remainders remain valid at divisor zero under this contract. Replacing an equal-valued subexpression preserves every enclosing operation by congruence. $\square$

6.3 Termination without a normal-form optimality claim

For an expression tree e , let $N(e)$ count operation and leaf nodes, with numeric literal payloads treated as part of a leaf, and let $A(e)$ count occurrences of word addition. Shared occurrences are counted as occurrences in the expanded tree. Put

$$\mu(e) = N(e) + A(e).$$

Proposition 6.4 (A decreasing measure). *Every rewrite step of the release decreases μ strictly. Thus any rewrite sequence has at most $\mu(e_0) - \mu(e_N)$ steps. In particular the unbudgeted normalization strategy terminates.*

Proof. The rule **disjoint-add** changes an addition node to an OR node without changing its children; it preserves N and decreases A by one. Every other rule replaces a term by a proper subterm or a single constant, hence strictly decreases N and introduces no addition. In an enclosing context the change in μ is unchanged. The nonnegative integer measure proves termination and the bound. $\square$

This statement does not claim confluence, a unique syntactic normal form, minimum proof length, or minimum depth. The producer uses a fixed rule order after recursively processing children. Its finite proof-step budget can stop a terminating computation early; the API then returns **fallback_required**. The extensional signature of Section 5, when applicable, provides canonical semantic information independently of syntactic choices.

6.4 Replay and the composition theorem

A certificate records the schema identifier, source-label fingerprints, entry, semantic contract, initial-expression fingerprint, a sequence of rule/path instructions, and a claimed final expression. For a target certificate it also records the nine rows and target name. The caller

supplies the source bundle and target again. The checker reparses that source, checks the bindings, replays every instruction, compares the obtained expression with the claimed one, and recomputes the target rows. A certificate cannot introduce an unproved assumption or substitute its own target for the caller’s target.

Theorem 6.5 (Source-to-verdict composition, K4). *Assume the frontend implements the source contract of Lemma 6.1, the checker implements the rules of Proposition 6.3, and replay successfully establishes $e_0 \rightsquigarrow e_N$. Then the source and e_N return the same value at every positive width on valid inputs. If e_N is an accepted pair and its recomputed table for the external target passes, the source transformer is sound at every positive width. If every table row is optimal, so is the source transformer. A failed row supplies a width-one refutation of the source.*

Proof. Induct on the replay instructions. Each instruction selects a valid subtree path and replaces its current subexpression by an instance of a value-preserving rule, establishing equivalence of successive whole expressions. Transitivity and Lemma 6.1 establish source equivalence with e_N . Apply Theorem 5.2 to the recomputed rows. Its concrete witness also refutes the source because normalization preserved the returned masks. $\square$

The producer’s search strategy is absent from this implication. The implementation’s checker does not run that search, although producer and checker share local-rule code. The trusted implementation comprises parsing, call expansion, schema validation, guards, local rules, replay, and the finite table evaluator. The mathematical proofs and finite regressions have not been replaced by a proof-assistant verification of that Python code. Nor does this theorem certify a separate compiler lowering of the source language.

6.5 Result semantics

Result	Meaning under the declared contract
certified	Target soundness at all positive widths; optimal reports the separate precision check.
unsound	Replayed normalization and a concrete width-one target counterexample.
normalized	Source-to-expression equality only; no target soundness claim.
fallback_required	Unsupported source/target/residual or exhausted budget; no complete target verdict.
invalid_certificate	A supplied certificate fails validation, binding, or replay.

Input and internal errors are separate failure outcomes. A normalization-only certificate has a null target and no rows; a target check does not accept it as a soundness certificate. Consumers must accept a transformer only from a successful target verification or check with status **certified**.

7 From consumer observations to whole-word reconstruction

7.1 The observation required by a consumer

Let D be the domain satisfying the checked path conditions of a source expression, and let $C : D \rightarrow Y$ denote its actual downstream consumer. An observation $\pi : D \rightarrow O$ is sufficient when

$$\ker \pi \subseteq \ker C. \tag{7.1}$$

The domain is part of the statement. A condition discovered in a branch cannot be used outside that branch without a new proof.

Proposition 7.1 (Consumer factorization, K6.1). *Condition (7.1) holds if and only if there is a unique map $\overline{C} : \pi(D) \rightarrow Y$ such that $C = \overline{C} \circ \pi$.*

Proof. Define $\overline{C}(\pi(x)) = C(x)$. Kernel inclusion is exactly the condition that this definition be independent of the representative. The equation gives existence, and surjectivity onto $\pi(D)$ gives uniqueness. Conversely, equality of observations implies equality after composition with $\overline{C}$. $\square$

This factorization criterion makes the implementation obligation explicit. An observation must be tied to the source value, its domain, and its consumer; a small observation type alone proves nothing. Research producers propose queries from actual uses of a result. Checkers reconstruct the source expression and path facts, validate the observation derivation, and replay its substitution at the designated occurrence.

A shift amount represented by modular word arithmetic need not equal the mathematical integer suggested by its syntax. Its action on a particular operand can nevertheless be determined by a small quotient of the amount. A comparison may need only a result sign under a checked no-overflow condition. The implemented families cover particular count, shift, support, sign, and result observations; a complete synthesis procedure for the best observation of an arbitrary program remains open.

7.2 A kernel does not label its classes

Recovering an entire word requires more than proving that an action forgets certain bits. For fixed $0 \leq s \leq w$, put

$$M_s = 2^{w-s} - 1, \quad \pi_s(x) = x \& M_s.$$

The kernel of left shift by s is the kernel of π_s . Every class has one representative among the submasks of M_s . Identifying a proposed inverse with that representative still requires a native semantic equation.

Proposition 7.2 (Labeled shift reconstruction, K6.2). *Under the declared total word semantics, for $0 \leq s \leq w$,*

$$\text{LSHR}_s(\text{SHL}_s(x)) = x \& M_s.$$

Consequently, if $x \& M_s = x$, this round trip returns x . The amount s is fixed in the original environment throughout both actions.

Proof. The left shift discards exactly the s highest bits and moves every remaining bit s positions upward. Logical right shift moves those bits back and fills the highest positions with zero. This is the stated projection; at $s = w$ the retained support is empty. The guarded conclusion follows by substitution. $\square$

The research artifact also derives the universal equation in its preceding symbolic proof system and records its instantiation, support premise, ground equality steps, and consumer substitution. The native projection law is an explicit proof dependency.

The distinction matters on a three-bit carrier. The maps $x \mapsto x \& 3$ and $x \mapsto \text{swap}_{0,1}(x \& 3)$ have the same kernel and agree at 0, 4, 3, but disagree at 1. Kernel information and those supplied values do not identify the internal labels. This is a countermodel to an insufficient interface, not an execution of the actual shift round trip.

7.3 Rows forced by atomic observations

For a finite observation set Q with at least two elements, use the named carrier $A = \mathcal{P}(Q)$ with union and intersection. A native cell action determines a map $\phi : Q \rightarrow Q$ and a guard $G \subseteq Q$. Its backward observation row is

$$h(P) = G \cap \phi^{-1}(P). \quad (7.2)$$

Lemma 7.3 (Atomic completion, K7.1). *The row in (7.2) preserves union and intersection. Its values on singleton subsets determine its complete table among all maps preserving these operations.*

Proof. Membership in either side of each preservation equation is equivalent to membership in G and the corresponding Boolean combination of $\phi(q) \in P$. Singletons generate nonempty subsets by unions; the intersection of two distinct singletons generates the empty set. Induction on these expressions proves uniqueness from the supplied atomic values. $\square$

This is a finite instance of the forced-row viewpoint of Paper I. In the examples below the supplied atoms generate the whole carrier: $S = D = A$, and the row kernel is $\kappa = K = \ker h$. A total realization on the same named carrier witnesses protection of its distinct central elements. Row kernels can be nontrivial while the central relation remains diagonal. No growth of the forced domain beyond the generated subalgebra is claimed here.

The completed tables are consumed by the transitions of the checker. The ground tables describe the named carrier; they do not assert lattice identities on every unnamed element of an arbitrary total model.

7.4 Finite gluing and its semantic bridge

Let $\delta : S \times B \rightarrow S$ act on observations and bit-column types, with initial observation s_0 and acceptable final observations $\text{Good} \subseteq S$. A certificate supplies a finite carrier and an edge table over every legal column.

Theorem 7.4 (Closed observation carrier, K7.2). *If the supplied carrier contains s_0 , is closed under every $\delta(\cdot, b)$, and every one of its states satisfies **Good**, then processing any finite list of columns from s_0 ends in **Good**. More generally, a relation between a concrete partial-word state and its observation that holds initially and is preserved by every local step holds after every finite list.*

Proof. Induction on the column list preserves membership in the closed carrier; its final-state condition gives the first assertion. The same induction preserves the simulation relation and gives the second. $\square$

Unreachable certificate states are harmless if they also satisfy the obligations. Minimality of the state count is unnecessary. A word theorem additionally needs completeness of the column alphabet, a bridge from comparisons to numerical word values, and a sign-boundary argument. A graph check without these bridges establishes only a statement about that graph.

The procedure identifies the consumer and domain, derives labeled native observations, completes compatible rows, searches for a closed carrier, checks its local and boundary obligations, and lifts the result through the semantic relation. Source binding and replay remain necessary at each substitution. It does not assume that every optimal abstract transformer preserves KnownBits join.

8 Signed words and joint mask–interval carriers

8.1 The carrier that must be preserved

For width $w \geq 1$, let m and a be compatible must/may masks, $m \subseteq a$, and let sgn_w interpret a raw word as a signed integer. Define

$$\mathcal{M}(m, a) = \{\text{sgn}_w(x) : x \& m = m, x \& \neg a = 0\}.$$

The joint carrier also has signed bounds and a zero flag:

$$J(m, a, L, U, Z) = \{x \in \mathcal{M}(m, a) : L \leq x \leq U, Z \vee x \neq 0\}. \quad (8.1)$$

An interval between mask extrema generally contains words outside the mask. Preserving those extrema alone does not establish preservation of (8.1). The Graal study keeps mask membership and order facts as distinct proof information.

The source is `IntegerStamp.java` [12], revision

3f5efeb49d934f8a0bcb3c115a6a28caa5622975.

The restricted source bridge checks actual helper bodies, loop directions, branches, guards, and returns. Supporting mask extrema and Java operations have declared native semantics and finite differential checks. This bridge has not been verified as a general Java compiler.

8.2 Descending repair needs a common higher context

Within one permitted sign half, let b be the minimum masked word. The descending pass visits optional nonsign positions from high to low and sets a position precisely when the trial word is at most U . At position i that word is

$$y_{>i} \ 1 \ b_{<i}.$$

The higher prefix is shared by every decision. Permitting a separate higher-prefix witness at each position admits combinations that no execution can produce.

The observation retains $P \subseteq R$, where $R = \{-1, 0, 1\}$ describes possible relations of the common higher prefix to the higher prefix of U . Write $p \triangleright q = p$ when $p \neq 0$, and q otherwise. For a cell, with lower comparison $r = \text{cmp}(b_{<i}, U_{<i})$, put

$$\phi(h) = h \triangleright \text{cmp}(y_i, U_i), \quad t(h) = h \triangleright \text{cmp}(1, U_i) \triangleright r.$$

The source predicate gives the guard G of higher relations selecting the proposed output bit. Updating P is exactly (7.2). The certificate assembles constraints upward while retaining unresolved higher context, describing the decisions of the downward execution.

The carrier $\mathcal{P}(R)$ has eight elements. Three supplied atoms force the other five cells of each of 24 row templates. The row kernels have two, four, or eight classes. The full observation is

$$(P, \text{cmp}(b, U), \text{cmp}(x, U), \text{cmp}(y, U), \text{cmp}(x, y)),$$

where comparisons refer to processed lower parts. Of at most $8 \cdot 3^4$ observations, the certificate supplies 49 states, 588 transitions over 12 legal column types, and 196 sign completions.

Proposition 8.1 (Descending masked maximum). *In the selected sign half, if a masked word $x \leq U$ exists, the descending pass returns a masked y with $y \leq U$ and $x \leq y$ for every such x .*

Proof. The initial word is the minimum of the sign half. Before each choice its unvisited optional bits are zero. Setting the current bit is feasible exactly when the trial word, the minimum completion of that proposed prefix, is at most U . If feasible, setting it maximizes the prefix; otherwise no admissible word with that prefix can set it. Induction over descending positions proves the result. The observation certificate records those same decisions through the shared P , and its complete transition and signed-end checks lift by Theorem 7.4. $\square$

Forgetting P is insufficient already at width two: the weakened interface permits output 1 for mask carrier $\{0, 1\}$ and $U = 0$. This separates that weakened interface from the required one; it does not establish minimality of all 49 states.

8.3 The complete upper helper

Put $C = \{x \in \mathcal{M}(m, a) : x \leq U, Z \vee x \neq 0\}$.

Theorem 8.2 (Upper contract, K8). *Under compatible masks, a typed signed upper bound, and the declared native source bridge, the modeled `computeUpperBound` returns $\max C$ when C is nonempty. Its empty branch is reached exactly when C is empty. Replacing U by the actual returned upper value preserves (8.1) for every additional lower bound L .*

Proof. Proposition 8.1 handles each invoked pass. The outer control selects sign halves and handles forbidden zero. Its comparisons involve seven named values: zero, relevant mask extrema, U , a pass result, and an arbitrary witness x . Every execution maps to their full weak order. All 47,293 ordered partitions of seven names are enumerated; the source IR is checked on the 744 cases consistent with necessary mask and pass facts. Membership is tracked by the provenance of returned values, in addition to their order.

In every admitted case a value return belongs to C , and any $x \in C$ forces that branch with $x \leq \text{result}$. These assertions establish maximality and characterize the empty branch. A nonempty maximum removes no word from an additional lower-bounded intersection. In the empty case the minimum-word return is at most the original typed U , so the tightened intersection stays empty. $\square$

The numerical minimum word can also be a genuine maximum. The extra empty-case observation distinguishes these situations; numerical equality to a sentinel is insufficient.

8.4 Ascending repair and the carry quotient

The lower helper first uses a descending pass. If its result g is below L , an ascending repair seeks the next masked word. Before a nonsign cell there are three reachable physical phases:

Phase	Increment started	Physical carry	Observed carry q
A	no	0	1
B	yes	0	0
C	yes	1	1

Before incrementing, $q = 1$ means a pending successor request; afterwards it means an unconsumed carry. Source-cell evaluation shows that A and C have identical output bits and next observation classes on every legal cell. Phase B is separated by an optional zero input.

A fixed bit remains fixed and passes q through. For an optional bit the source-derived action is

$$(g_i, q) \mapsto (g_i \text{ XOR } q, g_i \& q).$$

For each cell label and proposed output y , the backward row is

$$h_y(P) = \{q : \text{out}(q) = y, \text{ next}(q) \in P\}.$$

Eight such rows act on $\mathcal{P}(\{0, 1\})$. Their two supplied atomic values force the complete four-cell tables, from which the transition reconstructs its output and next carry.

The observation records q , comparisons $g/x, t/x, g/t$, two extremal-mask flags, and whether an optional bit occurred. Its closed carrier has eight states, 48 transitions over the six legal columns (m_i, a_i, g_i, x_i) , and 32 permitted sign completions.

Theorem 8.3 (Masked successor). *For any positive width, if an admissible masked word $x > g$ exists, the mathematical successor construction returns a masked word t satisfying*

$$g < t \leq x.$$

No positive-to-negative overflow occurs in that situation.

Proof. The source-cell factor supplies the local action, and the forced rows recover it. The certificate contains the empty-suffix observation and is closed under every legal nonsign column. Its sign checks establish the comparison and mask obligations. Comparison of lower parts is updated by letting a newly added higher bit take priority; the final signed bit reverses the order of its two values. These numerical relations and Theorem 7.4 prove the result for every list length, including an empty nonsign list. $\square$

Without nonsign optional bits, the virtual successor observation is still defined. The actual helper has a separate branch returning an intermediate zero. Relating the executed ascending loop to the sign carry uses the presence of an optional bit; the virtual convention is not substituted for that outer branch.

8.5 The lower helper has a conditional preservation contract

Theorem 8.4 (Lower contract, K9). *Assume compatible masks and typed signed L , and additionally*

$$L \leq 0 \quad \text{or} \quad a_{w-1} = 0. \tag{8.2}$$

Under the declared source bridge, the lower-helper result R satisfies $R \geq L$ and

$$x \in \mathcal{M}(m, a), \quad x \geq L, \quad (Z \vee x \neq 0) \implies R \leq x.$$

Thus replacing L by R preserves (8.1) for any U .

Proof. Fix such a witness x . If the minimum masked word already suffices, the corresponding source branch is safe. Otherwise the first pass gives the largest $g \leq L$ in its sign half. When $g < L$ and nonsign optional bits exist, Theorem 8.3 gives $g < t \leq x$. A successor in the same sign half cannot satisfy $t \leq L$, since that contradicts maximality of g . A change to the nonnegative half gives $t \geq 0 \geq L$ by (8.2).

Without nonsign optional bits, each permitted sign half is a singleton. A witness larger than the initial negative singleton forces a nonnegative half and makes the intermediate zero safe under (8.2). If zero is forbidden, a positive witness permits the source's least-positive masked correction: either the nonzero must mask or the least permitted power of two, no larger than x . The final guard ensures $R \geq L$, including cases without a witness.

The checker verifies the complete source control IR over seven named values. Their 47,293 weak orders and two Boolean flags give 189,172 combinations; 25,737 satisfy the necessary facts. Unused intermediate values stay unconstrained. Every real execution with a witness is therefore represented, and all admitted executions satisfy $L \leq R \leq x$. These inequalities give joint-carrier preservation. $\square$

The result need not be an exact minimum. At width eight, $m = 1$, $a = 129$, $L = -1$, and permitted zero, the helper returns 0, while the least feasible masked word is 1. Conversely, with $m = 0$, $a = 192$, and $L = 1$, it returns 127 although 64 is feasible. This second input violates (8.2); it explains why that condition cannot be dropped from the current contract. Neither helper example is a counterexample to the complete Graal constructor.

For an empty carrier, the contract imposes neither mask membership nor w -bit representability on the returned lower value. Consumers must process the empty outcome before reusing such a value as a typed bound. The complete `create` caller also updates temporary masks and passes through potentially incompatible states. The later source-bound caller profile now checks the corresponding joint-carrier obligations, as described below; it does not strengthen this standalone lower-helper theorem into an exact-minimum theorem.

8.6 Caller composition and the joint carrier

The subsequent profile in `research/create_joint/README_RU.md` binds the exact retained six-argument `IntegerStamp.create`, its factories, constructors, helpers, and iteration limit. For supported Graal widths 1, 8, 16, 32, 64, its Python certificate asserts that Empty is returned exactly for an empty joint carrier, that a nonempty result preserves that carrier and has exact signed extrema, and that the caller establishes each helper precondition.

The proof has four logical phases: entry, exact extrema, prefix-closed extrema, and stability. Within a fixed sign bucket, signed order agrees with nonsign payload order. The descending helper supplies an exact floor; if it is strictly below a bound that is at most the bucket maximum, that maximum witnesses existence of a nonwrapping successor. Common-prefix facts true at both exact extrema are true throughout the interval, so adding them preserves the carrier. At most two state-changing passes establish the normal form; the next pass confirms the fixed point. These are logical proof phases, not a claim that every concrete call executes three iterations.

The source-bound Python checker replays its helper certificates and trusts explicit caller rules for these mathematical steps. Separate Lean lemmas formalize masked cyclic successor, conditional floor-successor composition, and caller invariants (Section 11). In particular, the Lean third-pass result assumes that two passes have established Normal and that Normal is fixed; it does not establish those source premises automatically. The concrete Java frontend, source correspondence, and complete caller-to-Lean bridge remain outside the formal theorem. Small exact enumeration, an independent signed-order digit-DP, and native execution of the retained Java fixture provide additional implementation checks, not an extrapolation to a general Java theorem.

9 Inferred source interfaces and their checked execution

The observation principle of Section 7 now has executable producers and independent replay paths. Their scope is a restricted source model, not unrestricted Java. The important change is that a supplied factor can be replaced by a derivation explaining which distinctions are retained, why its labels are correct, and how every source continuation is covered.

9.1 Finite observation closure and source-derived questions

Let R be a finite reachable carrier, B an input alphabet, $T_b : R \rightarrow R$ its transitions, O_b its output labels, and F its terminal observation. Start with predicates describing F and O_b . If a question p is retained, its backward transfer along b is $p \circ T_b$. The kernel of all retained questions induces a partition. A candidate partition is sufficient when equal classes have equal

terminal labels and, for every b , equal output labels and successor classes.

Proposition 9.1 (Checked observable factor). *Suppose the supplied partition covers R , the initial class is correct, and the preceding local stability and label conditions hold. Executing its quotient yields the same output trace and terminal observation as the source model on every finite input word. If every distinct pair of classes has a checked distinguishing continuation, the quotient is the coarsest behavior-preserving quotient of this model for these observations.*

Proof. The initial states correspond. Stability preserves the correspondence at each input symbol, while output agreement preserves the emitted trace. Terminal agreement gives the final observation. A distinguishing continuation prevents any behavior-preserving quotient from identifying the corresponding classes. This proves the stated minimality relative to the fixed model and consumer, not minimality among arbitrary observation languages. $\square$

The general finite producer in `research/observations/REPORT_RU.md` recovers the two-class ascending factor from three physical phases without calling the old hand-written quotient. Replacing the repair action produces three classes; a delayed-output example requires three backward-transfer levels and four classes. These cases check that neither the expected number of classes nor an immediate-output heuristic determines the answer. The length of a distinguishing continuation is not Paper II’s term-depth horizon.

For restricted word expressions, `research/inference/README_RU.md` describes a second inference policy. Its finite question library comes from residual-coordinate types: arithmetic carry or constant-tail equalities, accumulated mismatch bits, persistent Boolean registers, and ascending offsets. The search starts with no questions, records a conflicting pair whenever stability or labeling fails, adds a library question separating that pair, and then performs deletion pruning. Replay reconstructs the residual semantics and library and checks the trace. This is automatic selection within an explicit language; it does not synthesize arbitrary arithmetic abstractions. Source-interface success and independent target success remain separate, including when a particular target bridge is unavailable.

9.2 Positive-width completion and independent consumers

The signed source bridge constructs states (h, r) , where r is a residual state and h records whether a bit has been read. The initial state has terminal label `not-a-word`; it is not assigned a Boolean value. Every positive-length trace has $h = \text{true}$ and uses the signed terminal semantics, with the last bit providing the sign. A certificate supplies earlier-parent reachability witnesses and a complete transition table. Replay checks every listed state, every source-derived transition, every label, and closure under both bit symbols. Reachability prevents extraneous states; closure and induction prevent omission of a source trace. See `research/signed_bridge/README_RU.md`.

The observation factor of this completion model retains the initial distinction. For example, the positive-power-of-two interface has five classes when `not-a-word` is included, whereas an earlier consumer with different initialization used four. These are different declared consumers, not contradictory minimization results.

Source-bound loading checks the model and observation proof before extracting atomic inverse-image rows. The runtime stores class labels, atomic row images, and source identifiers, without retaining source text, IR, residual states, or a direct transition table. For a class q and input symbol b , it recovers the unique output/successor pair (y, q') satisfying

$$q \in h_{b,y}(\{q'\}).$$

The checked coverage and disjointness of these atomic preimages establish uniqueness. Finite unions reconstruct $h_{b,y}(P)$, and induction gives source-equivalent execution for every admitted

positive length. The implementation and its API boundary are documented in `research/signed_runtime/README_RU.md`. A detached inspection view or receipt is not a replacement for the source proof.

An independent consumer is checked by a reachable product with its separately specified target machine. The source questions do not acquire their semantics from that target. A source-only result retains `target_checked:false`; only the completed target obligation can justify `certified`. The corresponding implementation is `research/signed_targets/README_RU.md`. These are conditional all-positive-width statements under the accepted modular source model; Java’s physical 32- and 64-bit executions form separate finite correspondence checks.

9.3 Avoiding unnecessary residual enumeration

A full residual carrier can be expensive before its trivial observable factor becomes apparent. In the earlier route, the tautology

$$(x = 2^{31}) \vee (x \neq 2^{31})$$

required 65 full residual states and exceeded a 64-state construction cap. Three subsequent mechanisms address distinct parts of this problem.

First, checked Boolean reduction and dependency slicing precede residual enumeration. The checker verifies local identities on the actual IR; purity and totality justify removing unused dependencies. It never substitutes the mathematical sign of a literal for its width-dependent modular interpretation. This route is described in `research/signed_reduction/README_RU.md`.

Second, a concrete negative witness can be checked directly against the source, target, guard, and width, without a complete positive factor. Bounded unsuccessful search establishes no universal result. In particular, a counterexample at width one need not be a Java-long counterexample; see `research/signed_witness/README_RU.md`.

Third, conditional residual descriptions have a checked coverage relation instead of merely representing a few sampled residual states. Each edge differentiates the live DAG by one bit, justifies accumulated mismatch conditions, replays local reduction, and checks the resulting successor description and terminal label. Local simulation and complete edge coverage lift to every continuation. The coverage route reduces equality with 2^{31} from the unavailable 65-state full carrier to 34 descriptions without increasing the cap. For equality with 2^k , the documented family uses $k + 3$ descriptions, reaching the cap at $k = 61$; $k = 62$ still exhausts the budget. These facts and limitations are recorded in `research/signed_coverage/README_RU.md`.

A checked immutable context can reuse the common source/coverage/observation proof across separately supplied targets, while a compact DAG shares exact repeated JSON structures in portable packets. The receiving process still checks the common evidence and every target. DAG sharing reduces repeated representation, not semantic obligations, and neither a cached verdict nor a hash establishes a new theorem. The lifecycle and transport contracts are in `research/signed_context/README_RU.md` and `research/signed_compact/README_RU.md`.

10 From initial rows to consumer-directed proof DAGs

An adequate whole-source interface is one way to answer a consumer. Another is to prove only its requested equality from native source facts. This section separates the exact algebraic kernels from the policies that acquire those facts and from the checked objects subsequently used by a client.

10.1 Initial pure-row semantics beyond atomic completion

The earlier atom-generated word interfaces have $S = D = A$. A separate checker now implements the two-sorted finite ground pure-row fragment of Paper I. Its input consists of finite native carriers A, B , their positive-arity operation tables, and supplied equations $\alpha(b, a) = c$. Repeated or conflicting supplied cells are allowed: the input need not define a partial function before quotienting. Compatibility is imposed on the named native tables; no additional composition law for the action follows merely from an operation on B .

The certificate establishes the least forced carrier quotient $Q = A/\theta$ by synchronous feedback. Every row sees the same quotient in a round. Positive derivations justify each merge, closure checks exclude missing consequences, and the last round is a common fixed point. On each resulting row, the checker verifies the generated domain S , its homomorphism h , the exact ambient congruence

$$\kappa = \text{Cg}_Q(\ker h), \quad \kappa \cap S^2 = \ker h,$$

and the forced carrier-valued domain $D = S^\kappa$. External classes remain external initial values; they are not silently assigned arbitrary elements of Q . Chronological ground derivations and complete finite separating models provide the two sides of the claimed interface partition. A separately checked chosen completion proves existence of that completion, not leastness of its quotient. The precise restrictions, provenance, and replay format are in `research/pure_rows/README_RU.md`.

A local source-bound pilot makes $D \supsetneq S$ operational. For one nonsign ascending cell on the three physical phases A, B, C , the four supplied inputs are

$$S = \{\emptyset, \{B\}, \{B, C\}, \{A, B, C\}\}.$$

Their outputs are respectively $\emptyset, \emptyset, \{A, C\}, \{A, C\}$. The kernel contains $\emptyset \sim \{B\}$ and $\{B, C\} \sim \{A, B, C\}$. Intersecting the latter pair with $\{A, B\}$ and using transitivity gives $\{A, B\} \sim \emptyset$, hence the forced value $h(\{A, B\}) = \emptyset$ outside S . Here $|S| = 4$ and $|D| = 8$. The native carrier is protected: identifying distinct physical phase observations is not permitted by the source bridge.

The bridge checks the actual local repair instructions and the union/intersection laws of their guarded inverse image. Its scope is one cell under its stated source context, not whole-word reachability. A direct Boolean/range proof on the same seed facts is shorter in this pilot, so the implemented forcing mechanism is retained as a verified option, not as a demonstrated cost improvement. See `research/source_forcing/README_RU.md`.

10.2 Typed ground queries and visibility

The ground-query input is a finite many-sorted signature, closed equations E , and ordered requested pairs O . The term DAG is exactly $\text{Sub}(E) \cup \text{Sub}(O)$; every node references earlier arguments and is type checked. A positive event either names an actual equation of E or applies congruence using paths made solely of earlier accepted events. Every argument, endpoint, and claimed depth is checked. Iterative dependency extraction preserves DAG sharing instead of expanding a term such as $f(t, t)$ into an exponential tree.

The ordinary entailment mode verifies the depth of the supplied proof without claiming it is minimal. In exact-threshold mode, a proof at depth d is accompanied by a separating model at $d - 1$ whenever that lower horizon contains the query. At a restricted horizon, a model certifies only nonvisibility from the active equations. At the full horizon it can certify nonentailment from E . Neither result asserts universal disequality. The model supplies total typed operations and checks all active equations, including those absent from the positive proof. The contract and conformance experiment are given in `research/ground_query/README_RU.md`.

10.3 Scoped native facts and an independent target

For a supported source s , guard G , and explicitly chosen width domain W , native rules establish closed equalities $E_{s,G,W}$. The word/Boolean terms retain their interpretation and source identity. Rules include checked low-bit propagation, elementary word identities, equality-to-mask implications, source-to-sign predicates, and consequences of the independently defined target abstraction under G . The source body is admitted before any shortcut based on an empty guard.

The consumer expression c is supplied independently. The ground certificate must prove the exact compiled obligation $s = c$. Its semantic implication is

$$(\forall e \in E_{s,G,W}, G \Rightarrow e) \wedge E_{s,G,W} \vdash s = c \implies G \Rightarrow s = c \quad (w \in W).$$

The checker recompiles the source, checks every native premise, and replays the ground proof. This positive route does not require the covered residual model. The all-positive mode refers to the declared modular semantics, while a fixed-width request states its width explicitly. Source unsupportedness, verified empty domain, concrete refutation, insufficient premises, and resource exhaustion remain distinct outcomes; see `research/source_query/README_RU.md`.

A planner chooses source questions from a restricted syntax-derived candidate policy and retries a consumer checkpoint as facts arrive. A failed finite model belongs to exactly its current premises and horizon; adding a fact invalidates its use as a model of the enlarged basis. Exhausting the candidate policy is not a proof that no adequate observations exist. Optional fallback constructs the retained covered source model and a new guarded target product, charging both the failed initial route and fallback. The ordinary route receives the same policy and native attempts but uses unfiltered congruence closure. It is a QKF implementation alternative, not an external solver. See `research/source_planner/README_RU.md`.

10.4 Closed lemmas, prepared contexts, and direct emission

A batch can share checked native premises E and add derived closed equalities E^+ . Each imported lemma carries its ground proof and depends only on the preceding checked prefix. Such a lemma is not a universally quantified rewrite schema: matching syntax does not authorize changing the source, guard, width, or native algebra. If an earlier lemma proves $s = p$, a later consumer $s = c$ may explicitly use a residual proof $p = c$; the complete result follows by transitivity. Removing this dependency would invalidate the shortened consumer proof. Cold loading rechecks every imported proof. The evidence and lifecycle are specified in `research/source_lemmas/README_RU.md`.

Prepared contexts bind immutable source, IR, guard, width, algebra, presentation, and obligation data within one build. Their admitted structures can be shared between search and checking instead of repeatedly parsed from JSON. These process-local objects do not travel as trusted receipts and do not authorize stale or cross-scope promotions. Direct emission then writes the final dependency DAG from the checked positive entries, avoiding the intermediate batch packet and its repeated export checks. Unresolved obligations retain their scoped-model fallback. These are changes in preparation and representation under the same proof obligations, documented in `research/prepared_context/README_RU.md` and `research/direct_emission/README_RU.md`.

10.5 An applicable checked summary

The experimental SDK provides `build`, `check`, `apply`, `assess`, `explain`, and explicit `refine`. An independent check of source, request, and packet yields an immutable summary. In the signed Boolean profile, applying it checks the guard and width and executes the established

target action without rerunning the source. A new consumer can be assessed against that action on the exact finite sign/capped-popcount abstraction. A changed source, guard, width, selection, or unsupported consumer requires new justification. The local physical-phase profile instead applies only already proved inverse images for its selected cell.

Portable proof dependencies remain available through export and explanation. A receiving process checks the packet anew. A lower model is not turned into an executable positive summary, and a mixed batch exposes only its actually proved guarantees. The runtime accepts widths up to 4096, whereas an all-positive mathematical certificate is not bounded by that execution API limit. The SDK scope is in `research/applicable_summary/README_RU.md`. It is an experimental checkout API; the installed coordinatewise `qkf` contract remains separate.

11 Lean verification and its remaining boundaries

11.1 The retained ascending pilot

The original pilot is a standalone Lean 4.33.0 project using the standard library without Mathlib [13, 14]. It formalizes the ascending mathematical fragment above. Certificate constants are exported from the saved research certificate as data; handwritten definitions and general lemmas determine what those constants must prove.

Module	Checked content
RowRules	Union/intersection on $\mathcal{P}(\{0, 1\})$, inductive forced-cell rules, soundness, atomic uniqueness, and row-kernel compatibility.
Gluing	Simulation and invariant preservation along every finite column list; soundness of a closed certificate.
Core, Data	Source-cell phases and their factor; the complete six-column alphabet; row, state, and edge tables.
Checked	Atomic agreement, row uniqueness, recovery of the native action from completed rows, closure, and sign conditions.
WordSemantics	Numerical words, lower-part bounds, signed comparisons, output-mask membership, and the all-width successor theorem.
SourceBridge	Equality of observed numerical states and output traces after any number of modeled nonsign source cells.
Audit	Expected axiom dependencies of the principal theorems, enforced at build time.

Theorem 11.1 (Lean pilot scope, K10). *The distributed Lean source proves the masked-successor result for every list of legal nonsign columns and each allowed sign completion satisfying its witness premise. It proves the source-word factor for the explicitly modeled nonsign repair iteration. Their names are*

`QKF.mask_successor_all_widths`
`QKF.source_word_factor.`

The length lemma identifies the complete signed width as the list length plus one.

Proof. The checked certificate supplies the initial state, every transition, and every sign obligation. General gluing yields the invariant for arbitrary lists. A separate relation connects observations to accumulated natural-number values and proves each lower part less than its power-of-two bound. Signed comparison translates the relational assertions into $g < t \leq x$, while trace induction establishes mask membership. For the source bridge, local phase factorization commutes with each numerical step and is lifted by the simulation theorem. These are the dependencies of the named Lean proofs. $\square$

Finite obligations use `decide`; elementary integer bridges use `omega`. The audit reports no axioms for forced-rule soundness and atomic row uniqueness. The main numerical theorem uses only the standard `propext` and `Quot.sound`; the source-word factor uses `propext`. No additional axioms, proof placeholders, `native_decide`, or `bv_decide` are used. Exact expected axiom lists are enforced by `#guard_msgs`.

The pilot does not formalize both foundational papers in full, the 49-state descending certificate, the outer lower-order argument, the complete Java helper, or the Python checker. Its source bridge concerns the stated operational cell model; translating actual Java and host-long arithmetic remains a separate obligation. Lean verification improves confidence in this fragment without changing whole-program coverage counts.

The package has been built from a fresh extracted copy, and three numerical examples are checked. Four isolated negative controls change an initial state, an edge, a row cell, and a proof; all are rejected at the corresponding certificate or axiom-audit gate. These controls exercise the build and trust checks; proof terms establish the general results.

11.2 Numeric targets and conditional caller rules

A separate pinned project, `research/lean_targets/MASKED_WORDS_RU.md`, proves an exact numeric cyclic-successor theorem from an accepted target certificate. Every legal competitor is represented by a bit list of the same length; the emitted result depends only on the input projection, not on the chosen competitor. The theorem `exact_cyclic_successor` states membership, the least strictly greater legal value when one exists, and return to the minimum otherwise, for every positive input length. Its two retained source factors have two source classes, nine joint states, and 54 transitions each. These figures describe that target interface, not a replacement of the eight-state certificate in the original pilot.

`research/lean_targets/CEILING_COMPOSITION_RU.md` formalizes the next composition. An exact floor below a bound, followed by an exact cyclic successor from precisely that floor, gives the least admissible value at or above the bound when the caller supplies the required extrema. The empty result is an explicit option distinct from numeric zero. The floor contract is a theorem hypothesis; the proof does not validate the descending Java region by assuming a saved Python verdict. Mask equality and reseeding preserve the legal numeric carrier. Caller lemmas in `research/lean_targets/CREATE_CALLER_RU.md` establish sign-bucket order, common-prefix preservation, extrema monotonicity, nonwrapping witnesses, and the conditional final-pass argument. Java parsing and the establishment of the source-level premises remain separate.

11.3 F1a: the decoded typed positive ground calculus

The standalone project `formal/ground/README_RU.md` supplies a computable Lean checker for positive chronological ground-DAG certificates. Lean checks backward node references, symbol existence, arity, sorts, equations, proof events, argument paths, all positive goals, and recomputed depths. Every congruence premise refers to the accepted earlier prefix. Unused events are also checked.

Theorem 11.2 (Formal ground soundness). *For an accepted decoded request and certificate, every accepted goal has equal endpoint values in every many-sorted interpretation satisfying its declared ground equations E . The two endpoints have the same sort and actual values in that carrier.*

Proof. `QKFGround.checkRaw_sound` combines DAG admission with the positive proof checker. `check_sound` proves preservation of equality for paths, congruence arguments, chronological

events, and goals in a total algebra. `typed_check_sound` specializes this to the many-sorted interpretation, and `eval_typed` shows that admitted terms evaluate to real values of the declared sort. `accepted_goal_values` packages the resulting endpoint guarantee. These are checked general Lean proofs, not deductions from a finite set of exported examples. $\square$

Empty sorts are permitted when the signature and interpretation allow them; no artificial default value is introduced for a sort. The theorem therefore does not mistake two failed evaluations for equal semantic values. Checked depth establishes validity at the reported horizon, not minimum depth. Negative models, exact-threshold minimality, search completeness, and source-rule truth are outside F1a. Its input is a Lean `Request`, not unparsed JSON bytes; the Python-to-Lean name mapping and exporter remain an adapter boundary.

11.4 F1b: chronological import and conservative extension

The project `formal/composition/README_RU.md` uses the preceding ground kernel unchanged. Its decoded packet contains a common signature, explicit native premises E , chronological entries, and independently specified consumer endpoints. A native entry refers to an actual member of E . A derived entry supplies a ground certificate whose axioms exactly match selected earlier equalities, in their declared order. A `via` reference requires a previously established $a = b$ and an exact residual proof $b = c$ to produce $a = c$. Consumers point to accepted equalities and repeat their exact endpoints.

Theorem 11.3 (Formal checked composition). *If the composition checker accepts, all imported equalities L , including imports unused by the final consumer, and all accepted consumer equalities hold in every interpretation satisfying the original E . In particular,*

$$\text{Models}(E \cup L) = \text{Models}(E).$$

For typed consumers, both endpoints have the same actual semantic value.

Proof. Induction over chronological entries uses ground soundness for each selected earlier basis. Explicit residual composition uses transitivity. Every model of E therefore satisfies the imported equalities, while every model of $E \cup L$ satisfies E . Exact consumer endpoints yield their stated equalities. The Lean results are `all_admitted_sound`, `conservative_extension`, `check_sound`, and `accepted_consumer_values` in the `QKFComposition` namespace. $\square$

Together F1a and F1b formalize the decoded typed positive proof calculus and checked-premise composition. Native facts remain explicit assumptions, carrying evidence and scope checked by Python. Source epoch, guard, width, cached-request identity, JSON decoding, frontend semantics, and interface-to-runtime correspondence are not established by these Lean theorems. Exact hashes bind the retained inputs without proving that remaining semantic correspondence.

The F1b validation retains 12 positive packets with 56 consumer obligations, including four original SDK packets from the ordinary/query and no-lemmas/reuse matrix. It audits 12 general theorems and 29 named example results and rejects 17 corrupted Lean packets. The accepted formal projects use Lean 4.33.0 and Std, with only the audited standard `propext` and `Quot.sound` dependencies where needed; proof-hole and extra-axiom controls test that boundary. Exact results and scope are retained in the F1b validation report and the F1a validation report, at their separate results-only commits. None of these counts enlarges the external program corpus.

12 Implementation profiles and claim traceability

12.1 Coordinatewise CLI

The installable Python package exposes inspection, normalization, target verification, and certificate checking. Its runtime needs only the standard library. Verification constructs a source expression and produces a rewrite trace. Checking reparses the source and replays the supplied trace without invoking normalization search. The two paths share primitive-rule code.

The certificate format remains `qkf-rewrite-v3`, API results use `qkf-result-v1`, and the semantic contract is

```
total-transfer-words-v2; positive-width;  
modular-arithmetic; SMT-total-div-shift;  
standard-counts; nonbottom-KB.
```

The accepted final coordinatewise targets remain AND, OR, and XOR. A resource limit or unsupported residual produces an explicit unresolved outcome. SHA-256 fingerprints bind the complete supplied source-label map, including helper definitions; they identify bytes, not authorship.

The original three Nice to Meet You examples retain their upstream provenance and MIT notices. AND normalizes to $(z_a \mid z_b, o_a \& o_b)$; OR has the dual optimal table. XOR expands the actual meet helper and uses a checked disjoint-support addition to recover its optimal table. The checker does not assign semantics from the name of a helper.

12.2 Research execution and summary profiles

The distribution preserves the earlier `research/knownbits` and `research/graal` capsules and their source identities. Replay restores saved evidence and checks it in fresh processes; regenerating search, native Java correspondence, or measurements is a separate action with separately documented dependencies. The historical POSIX capsules retain their environment requirements. The newer source-query and summary modules are tested on Python 3.10 and 3.12 and require a full checkout; they add no runtime dependency to the coordinatewise package.

The principal new entry points are source-interface inference and execution (Section 9), exact pure-row and typed ground queries, guarded source consumers, and `research.applicable_summary` (Section 10). The SDK retains signed `no_lemmas` and local-phase `direct_cell` defaults; derived-lemma reuse and forcing remain explicit options. The controlled comparisons did not silently change these defaults.

Certificate kinds have distinct scopes. `qkf-ground-query-certificate-v1` proves statements from a declared finite ground presentation. `qkf-source-lemma-packet-v1` additionally binds native premises and checked imports to a source context. The summary root `qkf-applicable-summary-v1`, transported by `qkf-json-proof-dag-v1`, supplies the evidence used by an applicable SDK object. Preparation and direct-emission changes preserve these public formats and the independent public checker. A valid certificate of one kind cannot be substituted for an obligation belonging to another.

Positive fresh replay excludes producer/search/SMT imports and does not execute the complete source program; native premises are still independently checked. A concrete refutation separately evaluates its supplied witness. Model size, source-state limits, proof-DAG limits, and runtime-width limits are implementation budgets with explicit failure outcomes, not premises of an extrapolation from tested widths. Immutable checked objects protect ordinary API use, not a hostile Python process capable of reflection or monkeypatching.

12.3 Formal projects and stable claims

The original `research/lean`, later `research/lean_targets`, and new `formal/ground` and `formal/composition` projects pin Lean separately from Python. F1b has a local dependency on F1a. Their build/audit scripts, exporters, positive examples, and negative controls preserve the scope described in Section 11. Table 1 retains the original claim identifiers and identifies the additions separately.

Claim	Artifact	Scope
K1–K2	Paper supplement: finite core and relation-matrix oracle	Exact finite ground interface and independent finite checks.
K3	Sparse prototype and representation theorem	Conditional coverage/fixed-point contract; historical pass-limit caveat retained.
K4–K5	<code>src/qkf_certifier</code>	Coordinatewise source replay and nine-row semantics.
K6–K7	Observation/row kernels and original Lean gluing	Labeled consumer observations and arbitrary-length composition.
K8–K9	Retained upper/lower Graal capsules	Exact upper model and conditional lower preservation; separate source contracts.
K10	<code>research/lean/QKF</code>	Original ascending mathematical fragment and nonsign cell model.
New source profiles	<code>research/inference</code> , signed interfaces, <code>create_joint</code>	Restricted source-derived factors and caller contracts with explicit Python trust boundaries.
New proof profiles	Pure rows, ground queries, source lemmas, summary SDK	Exact finite algebraic evidence and scoped source consumers.
F1a/F1b	<code>formal/ground</code> , <code>formal/composition</code>	Decoded typed positive calculus and conservative checked-lemma composition.

Table 1: Stable claims retain their original meaning; new profiles state their additional scope explicitly.

Artifact links identify the technical snapshot; measurement reports identify their individual engines and environments. Historical native executions, fresh certificate replay, formal builds, and release packaging checks are separate records. Bundling an experiment does not promote it to a larger source or semantic scope.

13 Evidence, counterexamples, and remaining coverage

13.1 Retained baseline evidence

Manuscript v1.0 audited the public 0.1.0a1 commit on Python 3.12.14 and x86-64 Linux. Its retained evidence includes 67 passing tests, 22,140 concrete comparisons of the original AND/OR/XOR programs at widths one through four, and an independent matrix-versus-ground comparison on 804 instances and 35,856 interface pairs. All 29 rewrite identifiers had witnesses, with 23,751 finite before/after comparisons and checks of the decreasing measure.

The finite classification checked all 512 Boolean row functions and all 3^9 valid output tables per target, confirming sound-class counts 64, 64, 16 and monotone counts 26, 26, 16. Of 680 source mutations, 270 were certified, 180 refuted at width one, and 230 required fallback. These remain evidence about the pinned coordinatewise profile. Versioned logs retain the original timings; this revision does not treat them as new performance measurements.

13.2 Broader research has several denominators

Dataset or profile	Result	Interpretation
Known LLVM transcriptions	11/11	A development/regression set, distinct from the original 39-program corpus.
GCC 15.2 specializations	9/9	Frozen external transfer study over the selected nine dispatcher specializations.
Original Nice to Meet You solutions	11/39	Complete source-bound proofs in the declared research contract.
Their individual components	104/411	Component proofs; not additional complete programs.
Five Graal procedures, control	1/5	Complete mask result for OR.
Five Graal procedures, joint	2/5	Complete mask results for AND and OR.

Table 2: These populations and proof obligations are not interchangeable.

On the original 39 solutions, the result-observation stage increased the preceding control from 9/39 complete programs and 87/411 components to 11/39 and 104/411. Add reached 13/13 components and UsubSat 16/16. All 411 source components were prepared; source-to-expression preparation alone is not target verification. The other 28 programs include 23 targets without the required research target adapter and five flag-related strengthened obligations. Refutations of those unconditional strengthened obligations do not establish errors in the upstream guarded contracts.

The historical GCC study froze its backend and selection before outcomes. It certified nine SSA specializations from GCC 15.2 and replayed them in separate processes. Composition reduced final certificate states from 2344 to 1070 at unchanged coverage. Accounting for all search attempts gives 2344 versus 1474 states; the smaller certificate count is not the full search cost. Z3 also solved all 72 selected finite-width obligations. These are transfer and structural results on that selected corpus.

13.3 Context and reconstruction evidence

The whole-word reconstruction study retained all claims of a normalization query while replacing a 4401-node subproof by 26 recorded steps, including the universal projection lemma, applicability conditions, ground steps, and substitution. The complete proof decreased from 4905 to 530 steps. On its 18 constructed evaluation sources, the new mechanism proved 11 intended positive cases, left two positive cases unsupported, and returned unresolved on five faulty cases for which separate finite counterexamples were retained. It was a constructed evaluation, not a blind external benchmark.

The evidence also preserves cases where observation machinery increased cost without reducing a proof. The three-bit equal-kernel/different-label separator in Section 7 and the two-bit shared-prefix separator in Section 8 identify concrete information that cannot be discarded. Neither is an upstream program bug.

13.4 Upper and lower contract checks

Check	Recorded result
Upper, all masks/bounds/zero flags at widths 1–6	111,972 cases agree with exact masked maximum and emptiness.
Upper, extracted Java helper	114,040 calls agree with the declared model.
Upper, additional lower bounds at widths 1–4	24,174 joint carriers preserved exactly.
Lower, all small typed cases at widths 1–6	111,972 cases: 76,104 admitted by the sign contract; 35,868 explicitly unsupported.
Lower, extracted Java helper	114,044 calls agree with the declared model.
Lean pilot	Clean build, fresh extraction build, axiom audit, three examples, and four rejected isolated corruptions.

The universal upper certificate additionally has wide mathematical specializations, including 128-, 256-, and 4096-bit cases. These are not Java runs at those widths. Upper and lower tables each describe their own experiment; their related input populations are not added together to inflate coverage. Among upper returns numerically equal to the minimum word, the record separates 37,049 empty cases from 963 genuine maxima.

The lower study retains conservative and out-of-contract behavior, including the examples following Theorem 8.4. A strict-comparison source variant was rejected by the reused floor lemma even though its 2232 checked native cases did not exhibit a bug. Such rejection marks the boundary of a proof interface; it is not by itself an unsoundness verdict.

13.5 What the SMT comparisons establish

One historical comparison used the same stored source-derived formulas for 39 programs at widths 32, 64, and 128, with three solvers and two repeats: 117 obligations and 702 executions. Its unconditional treatment of flag-related programs was later separated from artifact and LLVM-compatible assumptions. A subsequent 32-bit study varied both that contract and the encoding of counts, with a one-second solver budget.

The latter change was substantial. Under the artifact contract, balanced encoding gave 19 UNSAT results for Z3 and 18 for Bitwuzla out of 39, with no SAT results; their prefix encodings gave 13 each. Under the historical contract, balanced encoding gave 16 and 20 SAT results respectively. These runs show why source semantics and encoding must be held explicit before a ranking is meaningful. Their historical versions, budgets, and records remain attached to the reports.

QKF supplies parametric proofs for supported interfaces; these SMT runs ask separate fixed-width questions and did not retain solver proof objects. A useful local result was an all-width QKF proof for Smax when the six recorded 128-bit SMT attempts reached their limits, while at 32 bits conventional solvers could be faster. The evidence supports further controlled comparison and identifies specific successful interfaces. It does not establish a general performance ordering or coverage of arbitrary SMT problems.

13.6 Later interface and algebraic evaluations

The completed full-source frontend-applicability study obtained 0/12 accepted external methods across two Graal-specific frontends (24 admission attempts). All attempts stopped at source admission; no source-model search or target proof was reached, and no program was refuted. This deliberately selected, previously inspected sample measures applicability beyond the old frontend contract. Its final acquisition and CI record is PR14; `research/external/frozen_v1/REPORT_RU.md` preserves the earlier provisional run and must not be read as the final status.

Later extension work does not retroactively change this result. The later frozen Java external studies retain their own denominators: 1/15 in frozen-v2 and 2/33 in Run27. The two Run27 successes are two overloads of one body/algorithm group; the other 31 cases are capability or contract gaps, not source refutations. Later reuse of development methods is not a new holdout evaluation. The locked corpus and acquisition rules are preserved in `research/external/frozen_v2/PROTOCOL.md` and `research/external/frozen_v3/run27/README_RU.md`. These results neither replace nor increase the earlier 11/39 program and 104/411 component figures.

The source-interface acceptance study used 23 main development cases and five explicit controls, comparing the frozen route, the new row route, and a direct-cell control. The old route completed all 23 main cases with 15 certified and eight refuted; both new routes completed 21 with 14 certified and seven refuted, leaving two source-cap failures. Truth outcomes agreed on their shared completed population, and the new routes had identical source interfaces and target obligations. A smaller factor did not imply a smaller proof: one delayed case fell from 61 to 33 classes while its complete packet grew from 22,240 to 92,017 bytes. See `research/acceptance/REPORT_RU.md`. The later reduction and coverage mechanisms address the reported cap failures without retrospectively changing this experiment.

The pure-row evaluation comprises 16 named, 5120 exhaustive two-element, and 128 seeded mixed presentations: 5264 finite algebraic inputs, not Java programs. The ground-query conformance set contains 203 registered requests, including 195 Catalan profiles through height five and eight named variants. It compares each activation horizon against the retained independent full-space ground procedure; a separate finite rewrite search exhibits a congruence/rewrite-depth gap of 2/3. These are correctness and representation studies described in the respective module READMEs, not evidence of a general speed advantage.

13.7 Selecting facts and comparing equal deliveries

The causal study in `research/causal_comparison/REPORT_RU.md` includes 32 cases, 122 scheduled case/route pairs, six preregistered unsupported contracts, and 116 executed pairs. It records all 1044 attempts. Adaptive acquisition reduces native questions on the mask and low-bit families: respectively 2/20, 4/42, 13/23, and 8/36 adaptive/eager attempts on the four named positive source cases. The complete adaptive SDK nevertheless costs more than the simpler direct route on these singletons. The experiment therefore establishes selective acquisition without establishing a complete-path speedup from it.

Reuse lowers the SDK’s median build/check cost on the four $N = 16$ mask/parity and all/fixed-width series in that study. These targets are distinct expressions equivalent under the same guard, not 16 semantically independent properties. Smaller final dependencies and reduced export work contribute to the result; it is not an isolated advantage of the query backend. In the local physical-phase pilot, forcing obtains a genuine value in $D \setminus S$, but direct controls already need only three physical source-cell evaluations and cost less.

The next comparison holds SDK functionality equal. It distinguishes one applicable summary, one checked module serving equivalent consumers through `assess`, and independently checkable packets for every target. QKF’s ordinary-to-SDK route includes adaptation and checking costs; raw proof-only timings are reported separately. On each of Python 3.10 and 3.12, that route is cheaper than the default in all four registered singleton cases. The corresponding once-ordinary service and each-ordinary independent-packet routes are cheaper in all 13 rows of their respective contracts. These internal baselines are specified in `research/sdk_comparison/REPORT_RU.md` and `research/sdk_comparison/PR52_BASELINES.json`.

The service can build one sufficient summary and assess later equivalent consumers. This saving does not imply transfer to a different source, guard, width, or semantic contract. The

independent-packet clients in that experiment did not retain a cross-call semantic or exact cache; exact-repeat conclusions therefore do not exclude a cheaper client that simply retains the checked certificate. These delivery distinctions are necessary for interpreting reuse.

13.8 Prepared contexts and direct proof emission

The prepared-context experiment records, on each Python version, five cases, 38 case/route pairs, and 646 attempts. Corresponding old/new deliveries and native attempts agree. Preparation reduces repeated schema parsing and reconstruction; in the mask/all-width/four-goal audit, ground parses fall from 144 to 74 and research Python calls from 1,583,455 to 1,273,278. The Python 3.12 lifecycle medians improve by 9.03–17.31% across the five registered deliveries, with build traced-memory peaks increasing by approximately 1.6–5.5%. These are implementation tradeoffs under unchanged proof obligations; see `research/prepared_context/REPORT_RT_RU.md`.

Direct emission removes the intermediate positive native packet. Its experiment records six cases, 48 case/route pairs, and 816 attempts on each Python version. The 42 matched old/new comparisons, including two scoped fallbacks, preserve delivery/result bytes, native attempts, foundations, and checkpoint identities. Across ten positive environment comparisons, query/no-lemmas lifecycle medians decrease by 27.96–38.80% relative to the prepared-context route. These are ranges of separate comparisons, not a pooled estimate or confidence interval.

Delivery	Prepared	Direct emission	QKF ordinary
Mask, one summary	835.98	578.97	347.75
Mask/all widths, four-consumer service	3482.99	2165.81	434.58
Parity/fixed8, four-consumer service	3477.76	2156.00	409.86
Sixteen exact repeats	2942.70	2119.99	505.14
Mask, four independent packets	6478.41	4080.38	1439.30
Open power predicate, scoped fallback	629.47	634.19	361.84

Table 3: PR49 experiment: lifecycle median milliseconds on Python 3.12 within each registered job. Prepared and direct-emission columns use query/no-lemmas. The final column is QKF’s simpler registered ordinary-SDK control for the same delivery contract.

Table 3 is drawn from `research/direct_emission/REPORT_RT_RU.md`; raw samples and identities are retained in its `RESULTS.json`. The simpler ordinary-SDK control remains cheaper than each of the four new ordinary/query by no-lemmas/reuse configurations in all ten positive environments. Direct emission is therefore a successful optimization within QKF, while an additional advantage of the advanced query path over QKF’s simpler route remains unestablished. No default switch follows from these timings.

The later experiments use seven fresh timing samples, with audit and memory runs performed separately. Lifecycle includes build, delivery encoding, independent cold checking, applicable consumer assessment, export, and explanation. Repeated application is measured separately. Traced Python build allocations are not resident memory or receiver memory; Python call counts are not machine instructions. Different runners and interpreter versions are not pooled. These are instrumented development measurements, not compiler hot-path or external-generalization results.

13.9 Validation provenance

The committed reports retain engine identities, registrations, raw outcomes, manifests, and replay recipes. Formal validation records are described in Section 11. Preparing this manuscript does not itself rerun the historical searches, native executions, benchmarks, or Lean builds. Any

additional release smoke checks are listed separately in the accompanying release-validation record, so a packaging check cannot be mistaken for a new research measurement.

14 Development priorities and versioned guarantees

Four identifiers remain explicit: manuscript version, software/source identity, certificate schema, and semantic contract. Manuscript 3.0 describes the technical snapshot through PR51 and accompanies software 0.3.0a1 research-alpha preparation. Coordinatewise schemas remain unchanged; research profiles retain individual schemas and source bindings. A release tag or deposit identifier must refer to the actual published revision, while a concept DOI denotes an evolving record. The existing version-2 Paper III DOI must not be presented as a new version-3 identifier.

Several tasks that were future work in version 2 now have concrete implementations: restricted automatic observation inference, source-bound signed execution, joint-carrier caller checks, initial pure-row semantics, source-to-consumer ground proofs, checked lemmas, and applicable summaries. F1a/F1b add general formal soundness for decoded positive proofs and chronological composition. Each closes a specific obligation under its stated contract; none establishes an unrestricted source-language verifier.

The next implementation choice should preserve the useful simpler ordinary-SDK path while testing additional mechanisms against it under equal delivery contracts. The registered broader comparison remains future work. It must retain native policies, caches, complete first-proof costs, cold receiver checks, application/assessment distinctions, failures, and memory tradeoffs. A smaller factor, fewer questions, fewer calls, or fewer bytes is informative but does not alone establish lower total cost.

Observation inference remains incomplete beyond the current finite language and restricted residual models. The next expansion should be justified by a concrete consumer that current native rules cannot handle, with explicit source correspondence and a frozen external evaluation after the new engine is fixed. Known development successes should not be relabeled as unseen external coverage. The finite-ground query kernel already distinguishes proof visibility from insufficient premises; a new candidate language must preserve that distinction rather than interpreting every negative model as a source bug.

The next formal obligations are the native source rules actually used by successful consumers, the source-to-term adapter, and the interface-to-runtime correspondence. For the Graal line, the descending and concrete caller bridges remain separate from the conditional mathematical composition theorems. F1 does not resolve these by itself. Coverage, discovery effort, portable proof size, replay cost, runtime cost, native correspondence, and formalized semantic scope remain separate progress measures.

15 Relation to previous work

Nice to Meet You synthesizes practical MLIR abstract transformers and combines components by abstract meet [5, 6]. Its candidate validation uses Z3 over finite widths. QKF studies a parametric certificate route: source reconstruction and replay connect the program to a semantic interface, and a finite product or gluing argument lifts the interface check to every positive width. The original artifact, the total research contract, and LLVM’s guarded or poison-producing operations require explicit semantic correspondence when their verdicts are compared.

KnownBits and the related tristate-number domain are established representations [7, 8]. Their arithmetic properties motivate concrete native facts in this study. The contribution here

concerns the specified finite semantic classes, source-bound proof architecture, and the use of consumer observations and forced rows to compose word-level obligations. The fixed-point and finite-state checks themselves use standard finite algorithms.

Bit-width-independent SMT approaches seek proofs through integer arithmetic, uninterpreted functions, and quantified encodings [10]. QKF currently succeeds where an adequate finite observation carrier and its semantic bridge can be constructed. Alive2 addresses LLVM refinement through bounded translation validation [11]; its source model and obligations differ from the total transfer-expression contract here. The experiments do not establish that one of these approaches subsumes the others.

The algebraic organization is inherited from Papers I and II [1, 2]. The comparable-pair principal-congruence formula used in Section 4 is due to Martin-Maroto, Ricciardo, and de Polavieja [9]. Kernel factorization, inverse-image preservation, and induction are elementary ingredients. The applied step is to justify a particular native observation, retain its labels and domain, complete its rows under the applicable theory, and make those rows support the checked composition.

Lean supplies an external proof checker for the word fragments and the decoded typed proof/composition calculus [13, 14]. This reduces the trusted mathematical argument for those formalized fragments to its definitions, proof terms, and standard Lean assumptions. It does not automatically verify the surrounding parsers, Python implementations, or source-language bridges. The precise boundary is part of the result.

16 Conclusion

QKF now connects finite algebraic consequences to two executable proof paths: a checked source interface that can be reused by consumers, and a direct source-to-consumer equality proof from scoped native facts. The first retains labeled observational distinctions and composes their actions across words. The second records exactly how native premises and earlier closed lemmas support the requested result. Initial pure-row semantics supplies a separate exact account of forced values, including values outside the generated domain.

The original coordinatewise characterization and signed-bound contracts remain intact. Later caller profiles and numeric target theorems strengthen particular composition steps without claiming formal verification of arbitrary Java. Lean now also establishes soundness of decoded typed positive ground proofs and conservative chronological lemma import; source semantics, adapters, and execution bridges retain explicit obligations.

The measurements give a concrete engineering direction. Prepared contexts and direct emission remove significant repeated work, while QKF’s simpler ordinary-SDK route remains the stronger registered cost baseline. This supports continued development of useful checked summaries and disciplined selection among internal backends. Wider source coverage, richer observation languages, and an advantage attributable to the more elaborate query and forcing mechanisms remain questions for controlled experiments and further proofs.

A Semantic operations and the complete rule catalogue

A.1 Word operations used by the frontend

Let $M = 2^w - 1$. Integer literals are reduced modulo 2^w , and signed values use two’s-complement interpretation. Bitwise NOT is $M \text{ XOR } x$. Addition, subtraction, and multiplication are modular. Unsigned and signed minima/maxima use their respective orders.

For unsigned shift count c , logical shifts return zero if $c \geq w$; arithmetic right shift fills

with the sign bit, including for $c \geq w$. Clearing or setting c low/high bits affects $\min(c, w)$ positions. Sign-bit operations affect position $w - 1$. Leading/trailing zero/one counts are the number of consecutive matching bits from the chosen end, in $[0, w]$.

Unsigned division/remainder by zero return M and the dividend, respectively. Signed division truncates toward zero and is then reduced modulo width; division by zero returns one for a negative dividend and M otherwise. Signed remainder has the dividend's sign and returns the dividend at divisor zero. Comparisons `cmp0` through `cmp9` denote, in order, equality, inequality, signed $<$, $\leq$, $>$, $\geq$, and unsigned $<$, $\leq$, $>$, $\geq$. Boolean connectives operate on Boolean values; selection chooses a word branch by a Boolean condition. Pair construction and projection have their ordinary meanings.

These definitions specify a pure total word language. They are not a claim that an arbitrary LLVM/MLIR backend implements this contract. The exact contract identifier and source grammar are part of the release manifest.

A.2 All 29 identifiers

In the table, 0 and $\top$ denote zero and all-ones words, W denotes the width word, and 1 denotes the ordinary integer-one word. An identity with two operand orders is used only when the implementation permits both. Guards are those of Lemma 6.2.

Identifier	Accepted law or condition
<code>idempotent</code>	$x \circ x = x$ for bitwise AND/OR and signed/unsigned min/max.
<code>self-cancel</code>	$x \text{ XOR } x = 0$ and $x - x = 0$.
<code>zero-and</code>	AND or multiplication with either operand zero gives zero.
<code>ones-or</code>	OR with either operand all-ones gives all-ones.
<code>zero-identity</code>	OR, XOR, and addition with zero on either side; subtraction with zero on the right.
<code>ones-and</code>	AND with all-ones on either side returns the other operand.
<code>not-constant</code>	NOT interchanges zero and all-ones.
<code>not-not</code>	Double bitwise NOT is identity.
<code>disjoint-and</code>	$x \& y = 0$ when the disjoint-support guard succeeds.
<code>disjoint-add</code>	$x + y = x \mid y$ under the same guard.
<code>clear-width</code>	Clearing W low or high bits gives zero.
<code>count-constant</code>	A matching bit count on a constant word is W ; a nonmatching count is zero.
<code>clear-zero-count</code>	Clearing zero low/high bits is identity; clearing bits from zero gives zero.
<code>set-zero-count</code>	Setting zero low/high bits is identity.
<code>set-leading-included</code>	Setting <code>countl_one(x)</code> high bits of y is identity when support of x is included in support of y .
<code>umin-zero</code>	Unsigned minimum with zero gives zero.
<code>umax-ones</code>	Unsigned maximum with all-ones gives all-ones.
<code>remainder-zero</code>	Unsigned or signed remainder of zero gives zero, including a zero divisor.
<code>urem-one</code>	Unsigned remainder by integer one gives zero.
<code>udiv-one</code>	Unsigned division by integer one is identity.
<code>count-bound</code>	$W <_u \text{count}(x)$ is false and $W \geq_u \text{count}(x)$ is true.
<code>compare-reflexive</code>	Reflexive comparison is true for eq, signed/unsigned $\leq, \geq$; false for ne and strict comparisons.
<code>compare-unsigned-bound</code>	Strict comparisons beyond unsigned endpoints are false; inclusive comparisons from zero or to all-ones are true. Also $0 <_u x$ and $x >_u 0$ are true if the low-bit guard proves one.
<code>select-constant</code>	A constant Boolean condition selects its corresponding branch.
<code>select-same</code>	Equal branches remove a selection.

Identifier	Accepted law or condition
<code>bool-constant</code>	Evaluate two Boolean constants; false absorbs Boolean AND and true absorbs Boolean OR.
<code>set-low-one</code>	Setting one low bit of zero returns integer one.
<code>zero-shift</code>	Shifting zero in any supported direction returns zero.
<code>shift-zero-count</code>	Shifting by zero is identity.

A.3 Proof arguments by family

Bitwise identities follow separately at every bit; modular cancellation and zero multiplication follow in $\mathbb{Z}/2^w\mathbb{Z}$. Signed and unsigned min/max are idempotent because each is a minimum or maximum in a total order. No signed-order law is substituted for an unsigned-order law.

Disjoint addition is ordinary addition without carries: at each position at most one operand bit is one, and induction from carry zero keeps carry zero. Reduction modulo 2^w then gives OR. In the leading-bit rule, every leading one of x is already a one of y by support inclusion; setting these positions changes no value. These arguments use proved guards rather than sampled word instances.

The clear/set/count rules follow directly from the affected position sets and the definitions of counts. Every count is at most w , and the encoded width word is its exact unsigned integer value. The integer-one rules remain valid at width one. That does not make arbitrary uses of integer one coordinatewise.

Unsigned comparisons with zero and M use the extrema of B_w . An operand with low bit one is nonzero, hence strictly positive in unsigned order. Reflexive comparisons and Boolean/selection identities follow from their truth tables. Remainder of zero is zero for nonzero divisors and, by the specified total extension, also for zero divisors. Division or unsigned remainder by one has the indicated value. The shift identities also cover oversized counts because the shifted zero's sign is zero.

The rule catalogue specifies local semantic equalities. Checking that the implementation tests the matching syntax and side conditions is part of the code audit and release regressions. An altered rule set requires a renewed correspondence check before applying the old source-to-verdict theorem to a new executable.

References

- [1] L. Shcherbakov. *Initial semantics of incompletely specified actions: forced quotients, kernel saturation, and completion*. QKF project, Paper I, preprint v1.0, 2026. <https://doi.org/10.5281/zenodo.22736397>.
- [2] L. Shcherbakov. *Equality-visibility depth in equational presentations: finite-ground spectra, certificates, and semigroup space*. QKF project, Paper II, preprint v1.0, 2026. <https://doi.org/10.5281/zenodo.22736558>.
- [3] L. Shcherbakov. *QKF Certifier: Width-independent KnownBits certificates*. Software v0.1.0a1, 2026. MIT license. <https://github.com/Len989/QKF-Certifier/tree/f93561682319e8b911ed32c01583f2a496dc59fc>.
- [4] L. Shcherbakov. *QKF-Certifier: observation kernels and width-independent word proofs*. Paper III, preprint version 2.0, 13 September 2026. <https://doi.org/10.5281/zenodo.22736714>. This identifier refers to the preceding version, not the present revision.

- [5] X. Peng, D. Kennedy, Y. Fan, B. Greenman, J. Regehr, and L. D’Antoni. *Nice to Meet You: Synthesizing Practical MLIR Abstract Transformers*. POPL, 2026. <https://doi.org/10.1145/3776722>. Preprint: <https://arxiv.org/abs/2512.06442>.
- [6] X. Peng et al. *Artifact for Nice to Meet You: Synthesizing Practical Abstract Transformers for MLIR*. Version v2 (Updated README), 2025. <https://doi.org/10.5281/zenodo.17371668>. Example provenance: <https://github.com/Hatsunespica/xdsl-smt/tree/4ec509edab067d62c00f76778789170061ae299c>.
- [7] LLVM Project. *llvm::KnownBits structure reference*. Official documentation, accessed 12 September 2026. https://llvm.org/doxygen/structllvm_1_1KnownBits.html.
- [8] H. Vishwanathan, M. Shachnai, S. Narayana, and S. Nagarakatte. *Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers*. CGO, 2022. <https://doi.org/10.1109/CGO53902.2022.9741267>. <https://arxiv.org/abs/2105.05398>.
- [9] F. Martin-Maroto, A. Ricciardo, and G. G. de Polavieja. *The pairwise distributive law of semilattice congruences*. Algebra Universalis 87, article 29, 2026. <https://doi.org/10.1007/s00012-026-00942-x>.
- [10] A. Niemetz, M. Preiner, A. Reynolds, Y. Zohar, C. Barrett, and C. Tinelli. *Towards Bit-Width-Independent Proofs in SMT Solvers*. CADE, 2019. <https://arxiv.org/abs/1905.10434>.
- [11] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr. *Alive2: Bounded Translation Validation for LLVM*. PLDI, 2021. <https://doi.org/10.1145/3453483.3454030>.
- [12] Graal Project. *IntegerStamp.java*. Pinned source revision 3f5efeb49d934f8a0bcb3c115a6a28caa5622975. <https://github.com/oracle/graal/blob/3f5efeb49d934f8a0bcb3c115a6a28caa5622975/compiler/src/jdk.graal.compiler/src/jdk.graal.compiler/core/common/type/IntegerStamp.java>.
- [13] L. de Moura and S. Ullrich. *The Lean 4 Theorem Prover and Programming Language*. CADE, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.
- [14] Lean FRO. *The Lean Language Reference: axioms and proof validation*. Toolchain used: Lean 4.33.0. <https://lean-lang.org/doc/reference/latest/Axioms/>. Release: <https://github.com/leanprover/lean4/releases/tag/v4.33.0>.